

École Polytechnique Fédérale de Lausanne

Thermal properties of MOFs from molecular dynamics

Master Thesis

Lucas De Almeida Kloss

November 11, 2026

Acknowledgments

[Add acknowledgments here.]

Use of artificial intelligence

Artificial intelligence (AI) tools were used to assist with code generation and syntax correction in the project code and the LaTeX report.

Contents

Acknowledgments **1**
Use of artificial intelligence . . . 1

1 Scientific motivation and scope **4**

2 Theory **5**
2.1 Quantum Mechanics . . . 5
2.2 Density Functional Theory . . . 6
2.3 Molecular dynamics . . . 7
2.4 Heat Capacity . . . 11
2.5 Deep learning . . . 13
2.6 Point Edge Transformer . . . 15

3 Methods **20**
3.1 MOFs and guest molecules . . . 20
3.2 Structure preparation and initialization . . . 20
3.3 PET-MAD and PET-SOL . . . 21
3.4 NPT simulation . . . 22
3.5 Hybrid C_P . . . 22
3.6 Uncertainty quantification . . . 24

4 Results **27**
4.1 MOF-5 . . . 27
4.2 MOF-303 . . . 31
4.3 ZIF-8 . . . 32

5 Conclusion **34**

A Transformers and attention **35**
A.1 Tokenization and embedding . . . 35
A.2 Attention mechanism . . . 36
A.3 Multi-head attention . . . 37

B Sparse automatic differentiation **38**
B.1 The Jacobian, JVPs, and VJPs . . . 38
B.2 Recovering multiple columns with a single evaluation . . . 39
B.3 Graph coloring . . . 39
B.4 Decompression . . . 40
B.5 The Hessian and HVPs . . . 41
B.6 ASD in PET . . . 42
B.7 Hessian truncation . . . 43

Abstract

Methane-loaded MOF-5 heat capacity is estimated by combining classical NPT molecular dynamics with a loaded-system harmonic quantum correction from Hessians. PET-MAD and PET-SOL calculations at 50, 100, and 150 methane molecules give exploratory room-temperature values of about 0.93, 1.07–1.08, and 1.16 J/g/K. Sampling errors and paired last-layer prediction-rigidity (LLPR) model uncertainties are propagated separately. Production drift, imaginary modes, incomplete relaxation, and sensitivity to uncertainty propagation prevent a claim of convergence. Additional PET-MAD results cover CO_2 and H_2O in MOF-5 and MOF-303, and CO_2 in ZIF-8. Water-loaded trajectories show substantial enthalpy drift. Variation across quenched minima remains unestimated. Empty-framework Hessians provide separate references. Experimental comparisons are included for water-loaded MOF-303 and CO_2-loaded ZIF-8, with explicit loading and measurement-condition qualifications.

1 Scientific motivation and scope

Metal–organic frameworks (MOFs) are porous crystalline materials built from metal nodes and organic linkers. Their pore geometry and chemical composition can be varied to accommodate different guest molecules, making them attractive for gas storage, molecular separation, and water harvesting. These applications involve both the uptake of molecules and the exchange of heat: adsorption releases energy, while regeneration generally requires energy to remove the adsorbed guests. The heat capacity of the loaded material determines how much its temperature changes for a given heat input at fixed loading and contributes to the energy required to heat it during a working cycle. Calculating this property is therefore useful for connecting molecular structure to the thermal response of an adsorbent.

Guest adsorption also makes heat capacity a sensitive probe of microscopic motion. A loaded MOF combines framework vibrations, internal guest vibrations, and motion of molecules within the pores. Their contributions depend on temperature, loading, and interactions between the host and guests. High-frequency vibrations require quantum statistics, whereas soft framework motion and guest rearrangements can be strongly anharmonic. Capturing both effects at an affordable computational cost is the central challenge of this work [21].

This study evaluates a hybrid approach that combines classical molecular dynamics with a harmonic quantum correction, using the machine-learned potentials PET-MAD and PET-SOL for both contributions. Methane-loaded MOF-5 provides the initial test system and a connection to the study of Kapil *et al.* [21]. The objective is to assess the temperature and loading dependence of its heat capacity and establish the convergence and validity of the approach with PET. Calculations with carbon dioxide and water, and with MOF-303 and ZIF-8, test how pore chemistry and framework flexibility affect the thermal response. In particular, MOF-303 offers polar water-binding sites [8], while ZIF-8 provides flexible pore windows [9]. The results presented here cover methane-, carbon-dioxide-, and water-loaded MOF-5, carbon-dioxide- and water-loaded MOF-303, and carbon-dioxide-loaded ZIF-8.

2 Theory

2.1 Quantum Mechanics

In order to better understand the methods used in this work, we briefly review some important quantum mechanics concepts. These are necessary to understand the Born–Oppenheimer approximation, which is what allows us to run classical molecular dynamics simulations with a potential energy surface generated. But also density functional theory, which is the method used to generate the reference energies, forces, and stresses for the machine-learned potentials.

2.1.1 Schrödinger's equation

In (non-relativistic) quantum mechanics, the fundamental equation that describes a quantum system is the Schrödinger equation [1]. Let $\hat{\mathcal{H}}$ be a Hermitian operator called Hamiltonian, and $|\Psi\rangle$ a vector in a Hilbert space called a state vector. The Schrödinger equation can be written as an eigenvalue problem for the Hamiltonian operator

$$\hat{\mathcal{H}}|\Psi_n\rangle = E_n|\Psi_n\rangle, \tag{2.1}$$

where the eigenvalues E_n are real numbers and correspond to the energy of the state $|\Psi_n\rangle$. More generally, if $|\Psi(t)\rangle$ or $\hat{\mathcal{H}}(t)$ are time-dependent, the Schrödinger equation can be written as

$$i\hbar\frac{\partial}{\partial t}|\Psi(t)\rangle = \hat{\mathcal{H}}|\Psi(t)\rangle, \tag{2.2}$$

where $\hbar$ is the reduced Planck constant and i is the imaginary unit. The position representation is a way of expressing the state of a quantum system in terms of the position. For instance, consider the Hermitian position operator $\hat{\boldsymbol{x}}$. It posesses eigenstates $\hat{\boldsymbol{x}}|\boldsymbol{x}\rangle = \boldsymbol{x}|\boldsymbol{x}\rangle$, with eigenvalues $\boldsymbol{x}$, which is a real number corresponding to the position of the particle. If we project $|\Psi(t)\rangle$ onto $|\boldsymbol{x}\rangle$, we obtain the wavefunction $\Psi(\boldsymbol{x}, t) = \langle\boldsymbol{x}|\Psi(t)\rangle$, which is a complex-valued function of position and time. In this representation, the momentum operator $\hat{\boldsymbol{p}}$ is also a Hermitian operator, and it's given by $\hat{\boldsymbol{p}} = -i\hbar\nabla_{\boldsymbol{x}}$. If we write $\hat{\mathcal{H}} = \hat{p}^2/2m + \hat{V}$ in the position representation, we have $\hat{\mathcal{H}}(\boldsymbol{x}) = -\frac{\hbar^2}{2m}\nabla^2_{\boldsymbol{x}} + V(\boldsymbol{x})$. The Schrödinger equation in the position representation is then

$$i\hbar\frac{\partial\Psi(\boldsymbol{x}, t)}{\partial t} = \left[-\frac{\hbar^2}{2m}\nabla^2_{\boldsymbol{x}} + V(\boldsymbol{x})\right]\Psi(\boldsymbol{x}, t). \tag{2.3}$$

2.1.2 The Born–Oppenheimer approximation

Let us now consider a system of N_e electrons and N_n nuclei with coordinates $\boldsymbol{r}$ and $\boldsymbol{R}$ respectively. The Hamiltonian of this system in the position representation can be written as

$$\hat{\mathcal{H}}(\boldsymbol{r}, \boldsymbol{R}) = \hat{T}_\mathrm{e} + \hat{T}_\mathrm{n} + \hat{V}_\mathrm{ee}(\boldsymbol{r}) + \hat{V}_\mathrm{en}(\boldsymbol{r}; \boldsymbol{R}) + \hat{V}_\mathrm{nn}(\boldsymbol{R}), \tag{2.4}$$

where $\hat{T}_\mathrm{e}$ and $\hat{T}_\mathrm{n}$ are the kinetic energy operators for the electrons and nuclei, and $\hat{V}_\mathrm{ee}$, $\hat{V}_\mathrm{en}$, and $\hat{V}_\mathrm{nn}$ are the electron–electron, electron–nuclear, and nuclear–nuclear interaction potentials. Because the mass of the nuclei is much larger than that of the electrons, we can decouple the electronic and nuclear motions using the Born–Oppenheimer approximation [2]. This approximation consits in separating the electronic part of the Hamiltonian $\hat{\mathcal{H}}_\mathrm{el}(\boldsymbol{R}) = \hat{T}_\mathrm{e} + \hat{V}_\mathrm{ee} + \hat{V}_\mathrm{en}(\boldsymbol{R})$ from the

nuclear part of the Hamiltonian $\hat{\mathcal{H}}_{\text{nuc}} = \hat{T}_{\text{n}} + \hat{V}_{\text{nn}}(\boldsymbol{R})$. We then solve the electronic Schrödinger equation by treating $\boldsymbol{R}$ as a fixed parameter:

$$\left[-\sum_a \frac{\hbar^2}{2m_{\text{e}}} \nabla^2_{\boldsymbol{r}_a} + V_{\text{ee}}(\boldsymbol{r}) + V_{\text{en}}(\boldsymbol{r}; \boldsymbol{R}) \right] \psi_0(\boldsymbol{r}; \boldsymbol{R}) = \varepsilon_0(\boldsymbol{R}) \psi_0(\boldsymbol{r}; \boldsymbol{R}). \tag{2.5}$$

By solving this equation at $\boldsymbol{R}$, we obtain the electronic ground-state wavefunction $\psi_0(\boldsymbol{r}; \boldsymbol{R})$ and its ground-state energy $\varepsilon_0(\boldsymbol{R})$. We can then use $\varepsilon_0(\boldsymbol{R})$ in the potential of the nuclear system, which is given by $U(\boldsymbol{R}) = \varepsilon_0(\boldsymbol{R}) + V_{\text{nn}}(\boldsymbol{R})$. The nuclear Schrödinger equation is then given by

$$\left[-\sum_i \frac{\hbar^2}{2M_i} \nabla^2_{\boldsymbol{R}_i} + U(\boldsymbol{R}) \right] \chi_\nu(\boldsymbol{R}) = E_\nu \chi_\nu(\boldsymbol{R}). \tag{2.6}$$

When solving this equation, we obtain the nuclear wavefunction $\chi_\nu(\boldsymbol{R})$ and its energy E_ν. The total wavefunction of the system can then be approximated as a product of the electronic and nuclear wavefunctions

$$\Psi_\nu(\boldsymbol{r}, \boldsymbol{R}) \approx \psi_0(\boldsymbol{r}; \boldsymbol{R}) \chi_\nu(\boldsymbol{R}). \tag{2.7}$$

2.2 Density Functional Theory

Within the Born–Oppenheimer approximation, the electronic ground-state energy $\varepsilon_0(\boldsymbol{R})$ must be determined for fixed nuclear coordinates $\boldsymbol{R}$. Solving Eq. (2.5) directly is computationally demanding because the many-electron wavefunction depends on all electronic coordinates. Density functional theory (DFT) instead uses the electron density $n(\boldsymbol{r})$, a function of only three spatial coordinates [6, 7].

2.2.1 The Hohenberg–Kohn theorems

The Hohenberg–Kohn theorems establish that the ground-state density determines the external potential, up to an additive constant, and hence the electronic Hamiltonian. They also provide a variational principle: the ground-state density minimizes the electronic energy functional at fixed electron number [6]:

$$\varepsilon_0(\boldsymbol{R}) = \min_{n \text{ admissible}} \left\{ F[n] + \int v_{\text{ext}}(\boldsymbol{r}; \boldsymbol{R}) n(\boldsymbol{r}) \, \mathrm{d}\boldsymbol{r} \right\}, \qquad \int n(\boldsymbol{r}) \, \mathrm{d}\boldsymbol{r} = N_{\text{e}}. \tag{2.8}$$

Here v_{ext} is the electron–nuclear potential introduced in Section 2.1.2, and the universal functional $F[n]$ contains the electronic kinetic and electron–electron interaction energies. The theorems establish this density formulation but do not give a practical expression for $F[n]$.

2.2.2 The Kohn–Sham equations

The HK theorems do not provide a method for finding the ground-state density. The Kohn–Sham (KS) approach introduces a system of noninteracting electrons with the same $n(\boldsymbol{r})$ as the interacting system [7]

$$n(\boldsymbol{r}) = \sum_a f_a |\phi_a(\boldsymbol{r})|^2, \qquad \sum_a f_a = N_\mathrm{e}, \tag{2.9}$$

where $\phi_a(\boldsymbol{r})$ are the KS orbitals and f_a are their occupation numbers. The universal functional is written as $F[n] = T_\mathrm{s}[n] + E_\mathrm{H}[n] + E_\mathrm{xc}[n]$, where T_s is the noninteracting kinetic energy, E_H is the classical electrostatic energy of the electron density, and E_xc contains the remaining interaction and kinetic-energy contributions. Minimizing the energy with respect to the orbitals gives

$$\left[-\frac{\hbar^2}{2m_\mathrm{e}}\nabla^2 + v_\mathrm{ext}(\boldsymbol{r};\boldsymbol{R}) + v_\mathrm{H}[n](\boldsymbol{r}) + v_\mathrm{xc}[n](\boldsymbol{r})\right]\phi_a(\boldsymbol{r}) = \epsilon_a \phi_a(\boldsymbol{r}), \tag{2.10}$$

with $v_\mathrm{H} = \delta E_\mathrm{H}/\delta n$ and $v_\mathrm{xc} = \delta E_\mathrm{xc}/\delta n$. Because these potentials depend on the density, the equations are solved self-consistently by alternating orbital and density updates until convergence. The challenge to having good DFT results is to find a good approximation for the exchange-correlation functional $E_\mathrm{xc}[n]$.

2.3 Molecular dynamics

The potential $U(\boldsymbol{R}^N)$ appearing in the nuclear Schrödinger equation (2.6) is the Born-Oppenheimer potential energy surface. To know how the system evolves over time, we should solve this equation. However, because nuclei are much heavier than electrons, they can be treated as classical particles. Therefore, we can model their dynamics using classical mechanics, which is the basis of molecular dynamics (MD) simulations. Let $\boldsymbol{R}^N = (\boldsymbol{R}_1, \ldots, \boldsymbol{R}_N)$ and $\boldsymbol{P}^N = (\boldsymbol{P}_1, \ldots, \boldsymbol{P}_N)$ be the positions and momenta of the N nuclei in the system. The classical Hamiltonian is then given by

$$\mathcal{H}(\boldsymbol{R}^N, \boldsymbol{P}^N) = \sum_{i=1}^{N} \frac{|\boldsymbol{P}_i|^2}{2M_i} + U(\boldsymbol{R}^N), \tag{2.11}$$

Once we know the electronic ground-state energy $\varepsilon_0(\boldsymbol{R})$, the dynamics of the nuclei can be treated classically. Using Hamilton's equations, we can derive the equations of motion for the nuclei:

$$\dot{\boldsymbol{R}}_i = \frac{\partial \mathcal{H}}{\partial \boldsymbol{P}_i} = \frac{\boldsymbol{P}_i}{M_i}, \qquad \dot{\boldsymbol{P}}_i = -\frac{\partial \mathcal{H}}{\partial \boldsymbol{R}_i} = -\nabla_{\boldsymbol{R}_i} U(\boldsymbol{R}^N). \tag{2.12}$$

Since analytically solving a system of $6N$ coupled ordinary differential equations is impossible, we need to use numerical methods to integrate these equations of motion in order to find the trajectories of the nuclei. One of the most popular methods is the Runge–Kutta method, which is a family of iterative methods for solving ordinary differential equations. However, in molecular dynamics, the most commonly used method is the velocity Verlet algorithm [**?**]. The velocity Verlet algorithm is a symplectic integrator, which means that it preserves the geometric structure of the phase space and conserves energy over long timescales. By writing the velocities as $\boldsymbol{V}_i(t) = \dot{\boldsymbol{R}}_i(t)$ and accelarations as $\boldsymbol{A}_i(t) = \dot{\boldsymbol{P}}_i(t)/M_i$, we can update the positions of the particles using

$$\boldsymbol{R}_i(t + \Delta t) = \boldsymbol{R}_i(t) + \boldsymbol{V}_i(t)\Delta t + \frac{1}{2}\boldsymbol{A}_i(t)\Delta t^2, \tag{2.13}$$

Using this new position, we can calculate $\boldsymbol{A}_i(t + \Delta t)$ and update the velocities using

$$\boldsymbol{V}_i(t+\Delta t) = \boldsymbol{V}_i(t) + \frac{1}{2}\left[\boldsymbol{A}_i(t) + \boldsymbol{A}_i(t+\Delta t)\right]\Delta t. \tag{2.14}$$

If we want to simulate a system with constant N, V and E, we can use the velocity Verlet algorithm to integrate the equations of motion. However, in this project we want to simulate a system with constant N, P and T, which aims to reproduce the conditions of a laboratory experiment. We have therefore to make use of statistical mechanics to sample the appropriate ensemble.

2.3.1 Statistical Mechanics

The goal of statistical mechanics is to relate the microscopic properties of a system to its macroscopic behavior. In the context of molecular dynamics, we are interested in the properties of the NPT ensemble, which is characterized by constant number of particles N, pressure P_{ext}, and temperature T. The quantity of interest in this case is the enthalpy, which is defined as $H = E + PV$, where E is the internal energy, P is the pressure, and V is the volume. The internal energy E being the Hamiltonian $\mathcal{H}(\boldsymbol{R}^N, \boldsymbol{P}^N)$, the enthalpy is then given by

$$H(\boldsymbol{R}^N, \boldsymbol{P}^N) = \sum_{i=1}^{N} \frac{|\boldsymbol{P}_i|^2}{2m_i} + U(\boldsymbol{R}^N) + PV. \tag{2.15}$$

In the NPT ensemble, both internal energy and enthalpy fluctuate as the system exchanges heat and work with its surroundings. In this ensemble, a macroscopic observable A is given by the ensemble average of its microscopic observable $A(\boldsymbol{R}^N, \boldsymbol{P}^N)$ as

$$\langle A \rangle = \int_0^\infty \mathrm{d}V \int_{\text{cell}(V)} \mathrm{d}^{3N}R \int \mathrm{d}^{3N}P A(\boldsymbol{R}^N, \boldsymbol{P}^N) e^{-\beta[\mathcal{H}(\boldsymbol{R}^N, \boldsymbol{P}^N) + PV]}, \tag{2.16}$$

The particles are confined to a periodic simulation cell of volume V and shape $\boldsymbol{h}$. Additionally, we assume the ergodic hypothesis, which states that the infinite time average of an observable along a trajectory is equal to its ensemble average. So the integral (2.16) can be replaced by a time average along a sufficiently long trajectory

$$\langle A \rangle = \lim_{\tau \to \infty} \frac{1}{\tau} \int_0^\tau A\big(\boldsymbol{R}^N(t), \boldsymbol{P}^N(t)\big)\, \mathrm{d}t. \tag{2.17}$$

In practice, we can't run the simulation for ever, so we need to run the simulation for long enough to ensure that the system has reached equilibrium and explored the relevant phase space. We can then replace the integral by a finite sum over the trajectory frames, giving the time average

$$\langle A \rangle \approx \frac{1}{M} \sum_{j=1}^{M} A\big(\boldsymbol{R}^N(t_j), \boldsymbol{P}^N(t_j)\big), \tag{2.18}$$

2.3.2 Thermostat

A thermostat exchanges energy with the particles to sample a target temperature T. The equipartition theorem states that each independent quadratic degree of freedom contributes $k_{\text{B}}T/2$ to the mean energy [4]:

$$\left\langle \frac{ax^2}{2} \right\rangle = \frac{1}{2} k_{\mathrm{B}} T, \qquad a > 0. \tag{2.19}$$

For f active atomic momentum degrees of freedom, this gives

$$K = \sum_{i=1}^{N} \frac{|\boldsymbol{P}_i|^2}{2M_i}, \qquad \langle K \rangle = \frac{f}{2} k_{\mathrm{B}} T. \tag{2.20}$$

Thus

$$T = \frac{2\langle K \rangle}{f k_{\mathrm{B}}}, \qquad T_{\mathrm{kin}} = \frac{2K}{f k_{\mathrm{B}}}. \tag{2.21}$$

Here $f = 3N$ for unconstrained momenta, or $3N - 3$ when total linear momentum is removed. The instantaneous T_{kin} fluctuates around the target.

Nosé introduces a positive generalized coordinate s and its conjugate momentum p_s [35]. At fixed cell, the extended Hamiltonian is

$$\mathcal{H}_{\mathrm{N}}(\boldsymbol{R}^N, s; \boldsymbol{\pi}^N, p_s) = \sum_{i=1}^{N} \frac{|\boldsymbol{\pi}_i|^2}{2M_i s^2} + U(\boldsymbol{R}^N) + \frac{p_s^2}{2Q} + f k_{\mathrm{B}} T \ln s. \tag{2.22}$$

The generalized coordinates are $(\boldsymbol{R}^N, s)$ and the canonical momenta are $(\boldsymbol{\pi}^N, p_s)$; Q is the thermostat inertia. The coefficient f is chosen for canonical sampling in physical time. Hamilton's equations in Nosé's auxiliary time τ are

$$\frac{\mathrm{d}\boldsymbol{R}_i}{\mathrm{d}\tau} = \frac{\partial \mathcal{H}_{\mathrm{N}}}{\partial \boldsymbol{\pi}_i} = \frac{\boldsymbol{\pi}_i}{M_i s^2}, \tag{2.23}$$

$$\frac{\mathrm{d}\boldsymbol{\pi}_i}{\mathrm{d}\tau} = -\frac{\partial \mathcal{H}_{\mathrm{N}}}{\partial \boldsymbol{R}_i} = \boldsymbol{F}_i, \qquad \boldsymbol{F}_i = -\nabla_{\boldsymbol{R}_i} U, \tag{2.24}$$

$$\frac{\mathrm{d}s}{\mathrm{d}\tau} = \frac{\partial \mathcal{H}_{\mathrm{N}}}{\partial p_s} = \frac{p_s}{Q}, \tag{2.25}$$

$$\frac{\mathrm{d}p_s}{\mathrm{d}\tau} = -\frac{\partial \mathcal{H}_{\mathrm{N}}}{\partial s} = \sum_i \frac{|\boldsymbol{\pi}_i|^2}{M_i s^3} - \frac{f k_{\mathrm{B}} T}{s}. \tag{2.26}$$

Using physical time $\mathrm{d}t = \mathrm{d}\tau / s$, physical momenta $\boldsymbol{P}_i = \boldsymbol{\pi}_i / s$, and $\eta = \ln s$ gives the Nosé–Hoover equations [36]:

$$\dot{\boldsymbol{R}}_i = \frac{\boldsymbol{P}_i}{M_i}, \qquad \dot{\boldsymbol{P}}_i = \boldsymbol{F}_i - \dot{\eta} \boldsymbol{P}_i, \qquad Q \ddot{\eta} = 2K - f k_{\mathrm{B}} T. \tag{2.27}$$

Dots now denote physical-time derivatives. The thermostat removes excess kinetic energy by increasing the friction $\dot{\eta}$, and reverses this feedback when the kinetic energy is below its target.

The project uses a chain of three thermostat variables to improve sampling [37]. Introduce coordinates $\boldsymbol{\eta} = (\eta_1, \eta_2, \eta_3)$ and auxiliary momenta $p_{\eta_j} = Q_j \dot{\eta}_j$. Its conserved extended energy, often called the chain Hamiltonian, is

$$\begin{aligned} &\mathcal{H}_{\mathrm{NHC}}(\boldsymbol{R}^N, \eta_1, \eta_2, \eta_3; \boldsymbol{P}^N, p_{\eta_1}, p_{\eta_2}, p_{\eta_3}) \\ &\quad = K + U(\boldsymbol{R}^N) + \sum_{j=1}^{3} \frac{p_{\eta_j}^2}{2Q_j} + k_{\mathrm{B}} T (f\eta_1 + \eta_2 + \eta_3). \end{aligned} \tag{2.28}$$

Keeping only the first thermostat recovers the single Nosé–Hoover extended energy. In these physical-time variables the dynamics is noncanonical: ordinary Hamilton's equations applied to this energy alone would omit the friction couplings. The corresponding generalized equations are

$$\dot{\boldsymbol{R}}_i = \frac{\boldsymbol{P}_i}{M_i}, \qquad \dot{\boldsymbol{P}}_i = \boldsymbol{F}_i - \frac{p_{\eta_1}}{Q_1}\boldsymbol{P}_i, \tag{2.29}$$

$$\dot{\eta}_j = \frac{p_{\eta_j}}{Q_j}, \qquad j = 1, 2, 3, \tag{2.30}$$

$$\dot{p}_{\eta_1} = 2K - f k_{\mathrm{B}} T - \frac{p_{\eta_2}}{Q_2} p_{\eta_1}, \tag{2.31}$$

$$\dot{p}_{\eta_2} = \frac{p_{\eta_1}^2}{Q_1} - k_{\mathrm{B}} T - \frac{p_{\eta_3}}{Q_3} p_{\eta_2}, \tag{2.32}$$

$$\dot{p}_{\eta_3} = \frac{p_{\eta_2}^2}{Q_2} - k_{\mathrm{B}} T. \tag{2.33}$$

Each additional thermostat controls the kinetic energy of the preceding one. Equivalently, $\xi_j = \dot{\eta}_j = p_{\eta_j}/Q_j$ are the friction variables.

2.3.3 Barostat

A barostat on the other hand exchanges work with the simulation cell to keep the system at a target pressure P_{ext}. The barostat adjusts the simulation cell size and shape to maintain the desired pressure. This project uses the flexible-cell formulation of Shinoda, Shiga, and Mikami [34], combining Martyna–Tobias–Klein (MTK) pressure control [33] with Parrinello–Rahman strain dynamics, as implemented in LAMMPS `fix npt` [12]. The instantaneous pressure includes kinetic and configurational terms:

$$\boldsymbol{\Pi}_{\mathrm{int}} = \frac{1}{V}\left[\sum_i \frac{\boldsymbol{P}_i \otimes \boldsymbol{P}_i}{m_i} + \boldsymbol{W}_{\mathrm{vir}}\right], \qquad P_{\mathrm{int}} = \frac{1}{3}\operatorname{tr}\boldsymbol{\Pi}_{\mathrm{int}}, \tag{2.34}$$

where $\boldsymbol{W}_{\mathrm{vir}}$ is the configurational virial tensor. The feedback is illustrated by the isotropic MTK equations. Let $\nu = \dot{V}/(3V)$ be the linear expansion rate, W_b the fictitious barostat inertia, and ξ_b the first friction variable of the chain thermostatting the barostat. For an imposed pressure P_{ext},

$$\dot{\boldsymbol{R}}_i = \frac{\boldsymbol{P}_i}{m_i} + \nu \boldsymbol{R}_i, \qquad \dot{V} = 3\nu V, \tag{2.35}$$

$$\dot{\boldsymbol{P}}_i = \boldsymbol{F}_i - \left[\xi_1 + \left(1 + \frac{3}{f}\right)\nu\right]\boldsymbol{P}_i, \tag{2.36}$$

$$W_b \dot{\nu} = 3V(P_{\mathrm{int}} - P_{\mathrm{ext}}) + \frac{6K}{f} - \xi_b W_b \nu. \tag{2.37}$$

The pressure difference accelerates the cell outward or inward; cell motion rescales positions and momenta and exchanges mechanical work with the atoms. The $3/f$ momentum correction and $6K/f$ barostat term ensure the intended NPT statistical measure [33].

2.3.4 Trajectory analysis

When starting a MD simulation, the system is usually not in equilibrium. When monitoring observable quantities such as energy, volume, temperature, at first, they will have either too low or too high values. For instance, if we decide to start a simulation at a temperature of 300 K, the initial temperature might be much further away from this value. For this reason, we should let the simulation run until the system reaches equilibrium, that is when the observable quantities only fluctuate around their target values. We also want that after a long enough time, the system doesn't remember its initial state anymore. In other words, the system should reach the same equilibrium state regardless of its initial configuration. This assures that the quantities we measure don't depend on the initial configuration and are reproducible. One way to measure this is by computing the autocorrelation function of an observable A along the trajectory

$$C_A(k) = \langle (A_n - \langle A \rangle)(A_{n+k} - \langle A \rangle) \rangle, \qquad \rho_A(k) = C_A(k)/C_A(0). \tag{2.38}$$

These correlations determine how much independent information a trajectory contains. As k increases, the correlation $\rho_A(k)$ decays to zero, and the observable values become uncorrelated. The correlation time τ_A is defined as the time it takes for the correlation to decay to a negligible value

$$\tau_A = \Delta t \sum_{k=0}^{\infty} \rho_A(k). \tag{2.39}$$

2.4 Heat Capacity

The heat capacity of a material measures the energy required to raise its temperature by 1 K. In the context of MOFs, the heat capacity is an important property to understand their thermal behavior and stability. The heat capacity can be defined at constant volume C_V or at constant pressure C_P. In this project, we are interested in the heat capacity at constant pressure C_P, since it is what is measured in experiments and is relevant for practical applications. There are different ways to compute it depending on the level of theory used to describe the system. We show here the different methods used in this project.

2.4.1 C_P

As said earlier, the heat capacity is defined as the amount of energy required to raise the temperature of a system by 1 K. In the NPT ensemble, this can be expressed as the temperature derivative of the enthalpy

$$C_P(T) = \left(\frac{\partial \langle H \rangle}{\partial T} \right)_{N,P}, \qquad \langle H \rangle = \langle E + PV \rangle. \tag{2.40}$$

The average enthalpy $\langle H \rangle$ is obtained from the MD trajectory as a time average, as shown in Eq. (2.18). If we perform MD simulations at different temperatures, we can compute it by using finite differences

$$C_P(T) \simeq \frac{\langle H \rangle_{T+\Delta T} - \langle H \rangle_{T-\Delta T}}{2\Delta T}. \tag{2.41}$$

2.4.2 Dulong-Petit law

The heat capacity of a solid at constant volume can be estimated using the Dulong–Petit law. In a solid, the atoms are arranged in a lattice and can vibrate around their equilibrium positions. The potential energy of the system can be approximated as a quadratic function of the atomic displacements. The Hamiltonian is then a sum of quadratic terms in the atomic displacements and momenta

$$\mathcal{H}(\boldsymbol{R}^N, \boldsymbol{P}^N) = \sum_{i=1}^{N} \frac{|\boldsymbol{P}_i|^2}{2M_i} + \frac{1}{2}\sum_{i=1}^{N}(k_{i,x}R_{i,x}^2 + k_{i,y}R_{i,y}^2 + k_{i,z}R_{i,z}^2), \tag{2.42}$$

where $k_{i,x}$, $k_{i,y}$, and $k_{i,z}$ are the force constants for the i-th atom in the x, y, and z directions respectively. Using the equipartition theorem **??**, we can compute the contribution of each quadratic degree of freedom to the energy. We get that the average energy is $\langle E \rangle_{\text{har}} = 3Nk_{\text{B}}T$. For constant volume, the heat capacity is then given by

$$C_{V,\text{cl}}^{\text{har}} = \left(\frac{\partial \langle E \rangle_{\text{har}}}{\partial T}\right)_{N,V} = 3Nk_{\text{B}}. \tag{2.43}$$

In other words, the classical heat capacity of a solid is constant and doesn't depend on the temperature. This law doesn't hold well at lower temperatures, where quantum effects become important.

2.4.3 Einstein model

If we keep the same Hamiltonian as in the Dulong–Petit law, but now treat it quantum mechanically, we can compute the quantum heat capacity of a solid. In the Einstein model, we assume that all atoms vibrate independently with the same frequency ω. The Hamiltonian is then a sum of independent quantum harmonic oscillators

$$\hat{\mathcal{H}} = \sum_{i=1}^{N}\left(\frac{\hat{\boldsymbol{P}}_i^2}{2M_i} + \frac{1}{2}M_i\omega^2\hat{\boldsymbol{R}}_i^2\right). \tag{2.44}$$

The energy levels of a quantum harmonic oscillator are given by $E_n = \hbar\omega(n + 1/2)$, where n is a non-negative integer. Since these oscillators are independent, the total energy of the system is the sum of the energies of each oscillator

$$E = \sum_{i=1}^{N} E_i = \sum_{i=1}^{N} \hbar\omega\left(n_i + \frac{1}{2}\right). \tag{2.45}$$

Using quantum statistical mechanics, we can compute the average energy of the system by first computing the partition function

$$Z = \sum_{n_1=0}^{\infty} \cdots \sum_{n_N=0}^{\infty} e^{-\beta E} = \left(\frac{e^{-\beta\hbar\omega/2}}{1 - e^{-\beta\hbar\omega}}\right)^N. \tag{2.46}$$

The average energy is then given by

$$\langle E \rangle_{\text{Ein}} = -\frac{\partial \ln Z}{\partial \beta} = \frac{N\hbar\omega}{2} \coth\left(\frac{\beta\hbar\omega}{2}\right). \tag{2.47}$$

Using formula (2.40) again, the heat capacity is given by

$$C_{V,\text{Ein}} = Nk_{\text{B}} \left(\frac{\hbar\omega}{k_{\text{B}}T}\right)^2 \frac{e^{\hbar\omega/k_{\text{B}}T}}{\left(e^{\hbar\omega/k_{\text{B}}T} - 1\right)^2}. \tag{2.48}$$

We can compute the frequencies by computing the eigenvalues of the Hessian matrix of the potential energy surface. We can then insert them into the formula above to compute the quantum harmonic heat capacity of the system.

2.5 Deep learning

2.5.1 Neural networks

Deep learning uses neural networks with multiple learned layers to represent complex relationships between inputs and outputs. Neurons carry computed values; the trainable parameters are the connection weights and biases. As shown in Figure 1, each hidden layer forms weighted sums of the previous layer's outputs and applies a nonlinear activation:

$$\begin{aligned} \boldsymbol{z}^{(\ell+1)} &= \boldsymbol{W}^{(\ell)}\boldsymbol{h}^{(\ell)} + \boldsymbol{b}^{(\ell)}, \\ \boldsymbol{h}^{(\ell+1)} &= \sigma\left(\boldsymbol{z}^{(\ell+1)}\right), \qquad \boldsymbol{h}^{(0)} = \boldsymbol{x}. \end{aligned} \tag{2.49}$$

Here $\boldsymbol{z}$ denotes values before activation and $\boldsymbol{h}$ denotes the resulting features. The activation σ acts componentwise. Without nonlinear activations, composing these layers would remain an affine map. For L hidden layers and scalar regression, a final linear readout gives

$$\hat{y} = \boldsymbol{w}_{\text{out}}^{\text{T}}\boldsymbol{h}^{(L)} + b_{\text{out}}. \tag{2.50}$$

Evaluating these operations from input to output is the forward pass. The hidden layers build intermediate, or latent, representations rather than requiring every useful feature to be specified in advance. Suitable networks with sufficient capacity can approximate continuous functions on compact domains, but this representational ability alone does not guarantee successful training or generalization.

2.5.2 Backpropagation

Automatic differentiation (AD) computes derivatives of a numerical calculation by applying the chain rule to its elementary operations. Each operation, such as multiplication or an activation, has a known local derivative. AD combines these derivatives automatically, rather than requiring a hand-derived formula for the complete network [41].

A network with two hidden layers Consider the network in Figure 1: an input $\boldsymbol{x} \in \mathbb{R}^3$, hidden layers of four and three neurons, and a scalar output. All feature and bias vectors are

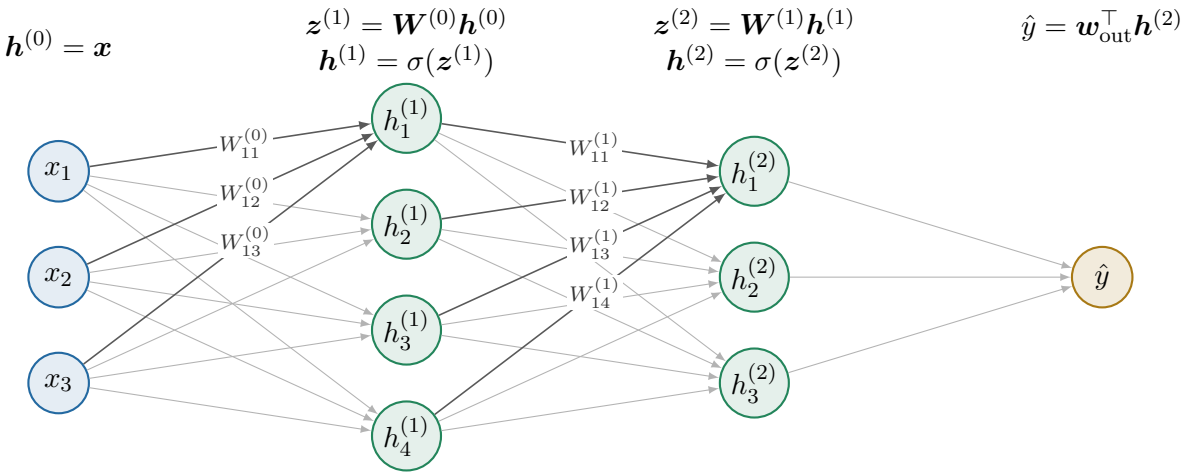


Figure 1: A simple fully connected neural network with three inputs, two hidden layers, and one scalar output $\hat{y}$. Circles represent input features or neurons, and connections represent learned weights between successive layers. Darker edges entering the first neuron of each hidden layer are labeled by $W_{1i}^{(0)}$ or $W_{1i}^{(1)}$; the other edge labels are omitted for clarity. Hidden layers apply nonlinear activations; the output is a linear readout. The layer equations above the network include the biases, whose separate connections are omitted for clarity. During training, the prediction and reference target y determine the loss $\mathcal{L}$. Backpropagation starts at this loss and computes parameter gradients for a subsequent optimizer update. This generic network illustrates the layer operations used within more specialized interatomic-potential architectures.

column vectors. With $\boldsymbol{h}^{(0)} = \boldsymbol{x}$, the forward pass is

$$\begin{aligned} \boldsymbol{z}^{(1)} &= \boldsymbol{W}^{(0)}\boldsymbol{x} + \boldsymbol{b}^{(0)}, & \boldsymbol{h}^{(1)} &= \tanh\big(\boldsymbol{z}^{(1)}\big), \\ \boldsymbol{z}^{(2)} &= \boldsymbol{W}^{(1)}\boldsymbol{h}^{(1)} + \boldsymbol{b}^{(1)}, & \boldsymbol{h}^{(2)} &= \tanh\big(\boldsymbol{z}^{(2)}\big), \\ \hat{y} &= \boldsymbol{w}_{\text{out}}^{\mathrm{T}}\boldsymbol{h}^{(2)} + b_{\text{out}}, & l &= \tfrac{1}{2}(\hat{y} - y)^2. \end{aligned} \tag{2.51}$$

Here $\boldsymbol{W}^{(0)} \in \mathbb{R}^{4\times 3}$, $\boldsymbol{W}^{(1)} \in \mathbb{R}^{3\times 4}$, and $\boldsymbol{w}_{\text{out}} \in \mathbb{R}^3$; tanh acts componentwise and y is the reference target. These matrix products, additions, and activations form a computational graph. The forward pass evaluates it and retains intermediate values needed for differentiation.

Reverse-mode AD and why it is effective Reverse-mode AD starts at the scalar loss and propagates sensitivities backward through the graph. For a vector $\boldsymbol{a}$, write $\bar{\boldsymbol{a}} = \nabla_{\boldsymbol{a}} l$. Starting with $\bar{l} = 1$, an operation $\boldsymbol{q} = f(\boldsymbol{a})$ contributes

$$\bar{\boldsymbol{a}} \mathrel{+}= \left(\frac{\partial \boldsymbol{q}}{\partial \boldsymbol{a}}\right)^{\mathrm{T}} \bar{\boldsymbol{q}}. \tag{2.52}$$

Sensitivities initially vanish except for $\bar{l}$; contributions add when a value influences the loss through multiple paths. AD evaluates these Jacobian-transpose–vector products using local derivative rules, without needing to assemble the full Jacobian.

For our network, let $r = \hat{y} - y$ and define $\boldsymbol{\delta}^{(k)} = \nabla_{\boldsymbol{z}^{(k)}} l$. The reverse pass is

$$\begin{aligned} \overline{\hat{y}} &= r, \\ \boldsymbol{\delta}^{(2)} &= (r\boldsymbol{w}_{\text{out}}) \odot \left[\mathbf{1} - \boldsymbol{h}^{(2)} \odot \boldsymbol{h}^{(2)}\right], \\ \boldsymbol{\delta}^{(1)} &= \left[(\boldsymbol{W}^{(1)})^{\mathrm{T}} \boldsymbol{\delta}^{(2)}\right] \odot \left[\mathbf{1} - \boldsymbol{h}^{(1)} \odot \boldsymbol{h}^{(1)}\right]. \end{aligned} \tag{2.53}$$

Here $\odot$ denotes componentwise multiplication and $\mathbf{1}$ is a vector of ones of the appropriate size. The bracketed factors are the derivatives of tanh. Transposed weight matrices carry sensitivities back to the preceding layer. The parameter gradients are

$$\begin{aligned} \nabla_{\boldsymbol{w}_{\text{out}}} l &= r\boldsymbol{h}^{(2)}, & \frac{\partial l}{\partial b_{\text{out}}} &= r, \\ \nabla_{\boldsymbol{W}^{(k)}} l &= \boldsymbol{\delta}^{(k+1)} (\boldsymbol{h}^{(k)})^{\mathrm{T}}, & \nabla_{\boldsymbol{b}^{(k)}} l &= \boldsymbol{\delta}^{(k+1)}, \qquad k = 0, 1. \end{aligned} \tag{2.54}$$

Each weight gradient is an outer product of a layer's sensitivity and its input; it has the same shape as the weight matrix.

The efficiency comes from reusing these intermediate values and sensitivities across all parameters. For a scalar loss, one forward and one reverse pass give the entire gradient, typically at a small constant multiple of the cost of evaluating the loss. Finite differences instead require perturbed evaluations for each parameter. AD avoids their step-size choice and truncation error, although floating-point roundoff remains. Its main tradeoff is memory for intermediate values; selected values can be recomputed to reduce storage [41].

Using AD for backpropagation Backpropagation is reverse-mode AD applied to a neural network's training loss: the equations above are its backward pass. For a batch, AD differentiates the averaged loss, including any regularization. An optimizer then updates the collection of weights and biases θ; for gradient descent,

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}, \tag{2.55}$$

where η is the learning rate. Training repeats the forward pass, backpropagation, and parameter update. Backpropagation computes the gradients; the optimizer performs the update. In this project, the pretrained potentials are used without further training, while AD also provides energy derivatives with respect to atomic coordinates for forces and Hessians.

2.6 Point Edge Transformer

A machine-learned interatomic potential (MLIP) approximates the energy of an atomic configuration using reference electronic-structure data. This reduces the cost of long trajectories and Hessian calculations. Reliable predictions require reference data that cover the configurations visited by the loaded framework. This section describes PET; a step-by-step explanation of its transformer operations is provided in Appendix A. Checkpoint and conversion details belong to Section 3.3.

Point Edge Transformer (PET) computes an energy by processing local atomic geometry, exchanging directed messages between environments, and summing learned contributions. The description below follows Section 6, Figure 2, and Appendix A of Ref. [31], using the paper's

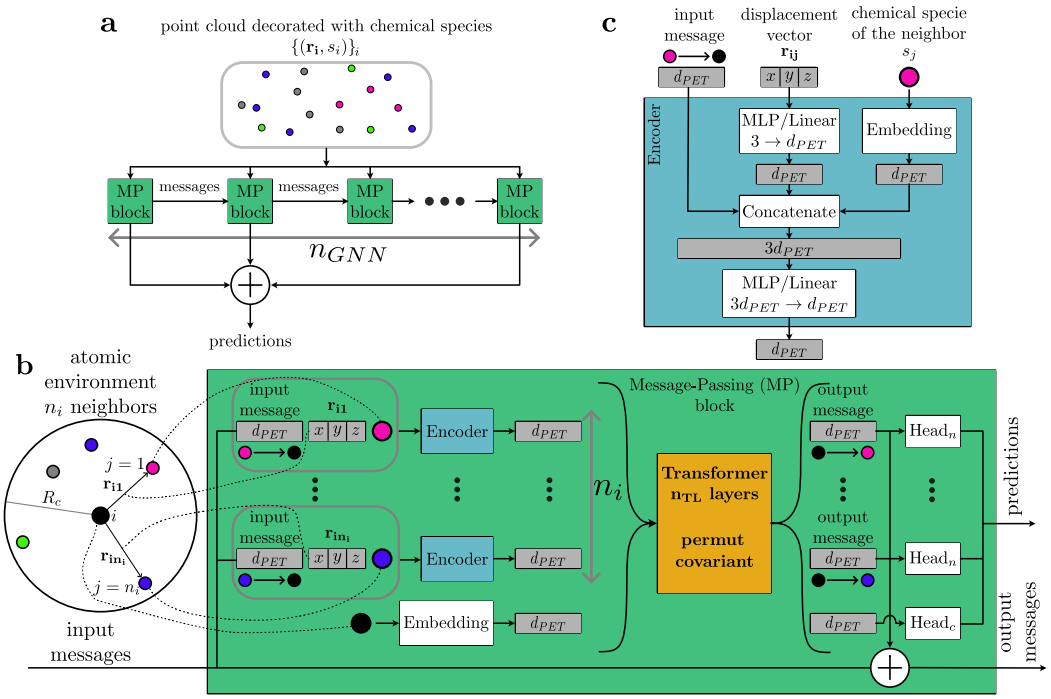


Figure 2: Point Edge Transformer architecture, reproduced from Ref. [31]. Each message-passing block embeds geometry, species, and incoming messages, processes the local tokens with a transformer, and produces both target contributions and outgoing directed messages. Local transformer depth n_{TL} and inter-environment message-passing depth n_{GNN} play different roles.

notation. The later PET-MAD potential [28] uses this architectural family; the exact checkpoint settings used here are given in Section 3.3.

Model overview The first step in the PET architecture is to define a local environment $A_i = \{(\mathbf{r}_{ij}, s_j) | r_{ij} < R_c\}$ for an atom i, where $\mathbf{r}_{ij}$ represents the position of atom j relative to atom i, s_j is the species of atom j, which is a categorical variable that could correspond to the the atomic number Z_j, and R_c is the cutoff radius, which limits the number of neighbors considered in the environment. The environment includes all neighboring atoms j within this cutoff distance from the central atom i. The same local block is applied to all atomic environments, usually batched for parallel evaluation, and the total energy sums their learned contributions. Once the environment is defined, the PET architecture processes it through n_{GNN} message-passing (MP) blocks as shown in Figure 2 a. The first MP block takes as input only the atomic environment A_i and produces an output (here the energy), as well as an output message. The next MP block takes as input the same atomic environment A_i as well as the output message from the previous block. It will then output a new energy and a new output message. This process is reapeated for all n_{GNN} MP blocks, with each block taking as input the same atomic environment A_i and the output message from the previous block. The final output of the PET architecture is the sum of all energy contributions from each MP block.

Encoding and embedding At each message-passing block k, PET converts the species, geometry, and incoming messages of the environment A_i into tokens of width d_{PET} (Figure 2b–c). Following Appendix A of Ref. [31], the central and neighbor species use separate learned embedding maps, $E_c^{(k)}(s_i), E_n^{(k)}(s_j) \in \mathbb{R}^{d_{\mathrm{PET}}}$. These maps allow the same chemical species to have

different representations in the central and neighbor roles. The relative position of neighbor j is embedded through a learned linear layer followed by a SiLU activation:

$$E_r^{(k)}(\boldsymbol{r}_{ij}) = \sigma\big(L^{(k)}(\boldsymbol{r}_{ij})\big)\,, \qquad L^{(k)} : \mathbb{R}^3 \longrightarrow \mathbb{R}^{d_{\mathrm{PET}}}. \tag{2.56}$$

This maps the three Cartesian components to the same feature width as the species embeddings. The resulting features encode spatial geometry and are not required to transform as physical vectors under rotations.

For blocks after the first, the neighbor also supplies the incoming message $\tilde{x}_{ij}^{(k-1)} \in \mathbb{R}^{d_{\mathrm{PET}}}$, sent from j to i in the preceding block. Equation (5) of Ref. [31] combines this message with the species and position embeddings to form the neighbor token:

$$t_j^{(k)} = M^{(k)}\Big[\mathrm{concatenate}\Big(\tilde{x}_{ij}^{(k-1)}, E_n^{(k)}(s_j), E_r^{(k)}(\boldsymbol{r}_{ij})\Big)\Big]\,. \tag{2.57}$$

Concatenation produces $3d_{\mathrm{PET}}$ components. The one-hidden-layer MLP $M^{(k)}$ mixes these inputs and compresses them to a token of width d_{PET}. The central atom i is implicit in the notation $t_j^{(k)}$.

In the first block, no incoming messages exist, so the message input is omitted and $M^{(1)}$ maps $2d_{\mathrm{PET}}$ features to d_{PET}. The central token is simply $E_c^{(k)}(s_i)$ and has no displacement input. An environment containing m neighbors therefore supplies $m+1$ tokens to the transformer: one for each neighbor and one for the center.

Appendix A also describes an alternative that initializes messages with species embeddings and omits the separate neighbor-species input, giving $2d_{\mathrm{PET}}$ inputs at every block. This is an alternative to the token construction above, rather than an additional encoding step.

Transformer layers and attention The tokens constructed above are processed by n_{TL} transformer layers. As explained in Appendix A, self-attention lets each token use information from every other token in the environment. Consequently, a neighbor's representation can depend on the surrounding geometry as well as its own displacement from the center.

A neighbor must lose its influence smoothly as it approaches R_c. Otherwise, removing its token at the cutoff would abruptly change the attention weights and their normalization, potentially making the predicted energy discontinuous. PET therefore introduces a smooth cutoff, given in Appendix F.1, Eq. (9), of Ref. [31]:

$$f_c(r \mid R_c, \Delta_{R_c}) = \begin{cases} 1, & r \le R_c - \Delta_{R_c}, \\ \frac{1}{2}\left[\tanh\left(\frac{1}{x+1} + \frac{1}{x-1}\right) + 1\right], & R_c - \Delta_{R_c} < r < R_c, \\ 0, & r \ge R_c, \end{cases}$$
$$x = 2\frac{r - R_c + 0.5\Delta_{R_c}}{\Delta_{R_c}}. \tag{2.58}$$

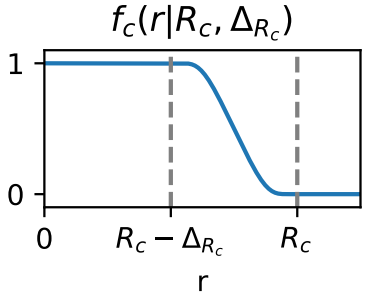


Figure 3: Smooth cutoff function $f_c(r \mid R_c, \Delta_{R_c})$ from Eq. (2.58) with $R_c = 5.0$ Å and $\Delta_{R_c} = 0.5$ Å. The function is unity for $r < R_c - \Delta_{R_c}$, smoothly decreases to zero over the switching interval, and is zero for $r > R_c$.

The width $\Delta_{R_c} > 0$ controls the interval over which a neighbor is switched off. The constant branches are used at the endpoints, where the fractions in the middle branch are undefined. The function joins smoothly to these branches, although a narrow switching interval can still produce large derivatives within it.

Section 6 of Ref. [31] applies this cutoff to the ordinary softmax attention weights and then renormalizes them. Let a, b, c index tokens in the environment centered on atom i, with $c_b = f_c(r_{ib} \mid R_c, \Delta_{R_c})$ for a neighbor and $c_b = 1$ for the central token. For one attention head, the modified weights and output are

$$\alpha_{ab}^{\text{cut}} = \frac{\alpha_{ab} c_b}{\sum_c \alpha_{ac} c_c}, \qquad \boldsymbol{y}_a = \sum_b \alpha_{ab}^{\text{cut}} \boldsymbol{v}_b, \tag{2.59}$$

where α_{ab} are the ordinary weights from Eq. (A.2) and $\boldsymbol{v}_b$ is the value vector. The superscript cut distinguishes the modified weights from the original ones. As a neighbor leaves, it disappears smoothly from both the weighted sum and its normalization. The central token retains unit cutoff weight, keeping the denominator positive in exact arithmetic.

Each transformer layer combines multi-head attention with token-wise feedforward operations, residual connections, and layer normalization; Eq. (**??**) illustrates the general arrangement. The output tokens of one layer become the inputs to the next, adding processing within the same environment. Shared operations preserve permutation equivariance: reordering the neighbor tokens produces the same reordering of their outputs, so their association with individual neighbors is retained (Figure 2b).

Transformer outputs The n_{TL} transformer layers produce $n_{\text{i+1}}$ outputs, $x_i^{(k)}$ for the central token and $x_{ji}^{(k)}$ for each neighbor j. The $x_{ji}^{(k)}$ correspond to the output messages that will be fed into the next MP block. The energy prediction of the block is computed using $H_n^{(k)}(x_{ji}^{(k)})$ for the neighbor contributions and $H_c^{(k)}(x_i^{(k)})$ for the central contribution, which are learned readouts that output scalar values. Using the cutoff function $f_c(r_{ij} \mid R_c, \Delta_{R_c})$ to smoothly remove contributions from neighbors beyond the cutoff distance, the neighbor contributions are summed to produce the total energy contribution from the block:

$$y_i^{\text{bond}\,(k)} = \sum_{j \in A_i} H_n^{(k)}(x_{ji}^{(k)}) f_c(r_{ij} \mid R_c, \Delta_{R_c}). \tag{2.60}$$

The cutoff is required here as well as inside attention. Attention removes a neighbor's influence on surviving tokens, but its own output need not vanish; the readout cutoff removes its direct contribution to the energy. The alternative in the paper's Eq. (6) is

$$y_i^{\text{bond}\,(k)} = \frac{\sum_{j \in A_i} H_n^{(k)}(x_{ji}^{(k)}) f_c(r_{ij} \mid R_c, \Delta_{R_c})}{\sum_{j \in A_i} f_c(r_{ij} \mid R_c, \Delta_{R_c})}. \tag{2.61}$$

This weighted average changes sensitivity to coordination and requires special handling when all neighbor weights vanish. The sum naturally returns zero for an empty neighbor list. The two readouts are alternatives that must be selected consistently in training and inference.

Appendix B adds fitted atomic self-contributions to the network output. Writing $E_{\text{self}} = \sum_i b_{s_i}$ for that composition-dependent baseline, the following expression assembles the paper's central, edge, and baseline terms:

$$\tilde{E} = E_{\text{self}} + \sum_{k=1}^{n_{\text{GNN}}} \sum_{i} \left[H_c^{(k)}(x_i^{(k)}) + y_i^{\text{bond}\,(k)} \right]. \tag{2.62}$$

Equation (2.62) is bookkeeping assembled from the text, not a separately numbered equation in Ref. [31]; E_{self} and b_s are explanatory symbols. The total $\tilde{E}$ is the learned potential U_θ used elsewhere in this report. Atomic self-contributions are fitted by linear regression on species counts before network training, subtracted from the targets during training, and restored at inference. They have zero Cartesian derivatives at fixed composition.

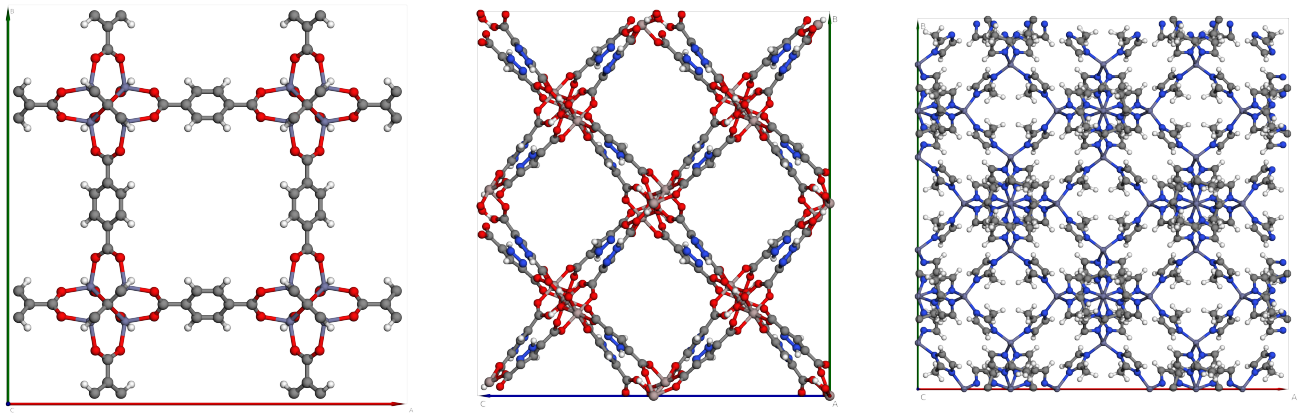
All blocks contribute to the readout. Central and edge terms are learned parts of the total energy, not uniquely defined physical atomic or bond energies. An edge output depends on its environment and incoming messages, so it represents a many-body contribution; no extra pair-counting factor of one half is implied. Forces follow by differentiating the total energy through all environments, messages, attention normalizations, and cutoffs, as in Eq. (**??**).

3 Methods

3.1 MOFs and guest molecules

The methane comparison uses the conventional cubic MOF-5 cell, with initial lattice parameter 25.866 Å and composition $C_{192}H_{96}O_{104}Zn_{32}$ (424 host atoms). Its Zn_4O nodes are connected by benzene-1,4-dicarboxylate linkers. Adding 50, 100, or 150 methane molecules gives 674, 924, or 1174 atoms, respectively. Each loading is simulated with PET-MAD and PET-SOL at eleven temperatures, with one trajectory per model, loading, and temperature, giving 66 trajectories in total. Periodic boundary conditions apply in all three directions. All host and guest atoms move freely; no rigid-molecule or fixed-framework constraints are imposed.

The additional starting structures in Figure 4 extend this comparison to other hosts. MOF-303, Al(OH)(PZDC), contains aluminium–oxygen rods connected by 1H-pyrazole-3,5-dicarboxylate linkers [8]; its $2 \times 2 \times 2$ host supercell contains 1024 atoms. ZIF-8, $Zn(mIm)_2$, contains zinc centres joined by 2-methylimidazolate linkers in a sodalite-type network [9]; its $2 \times 2 \times 2$ host supercell contains 2208 atoms. The additional PET-MAD campaigns contain 50 CO_2 or 50 H_2O molecules in MOF-5 and MOF-303, and 54 CO_2 molecules in ZIF-8. They add 55 trajectories on the same temperature grid, giving 121 runs in total. Each uses 500 ps of flexible-cell NPT at 1 bar, with a 0.5 fs timestep and the first 100 ps discarded. The methane-specific settings below are retained; additional relaxation and Hessian diagnostics are given in Appendix **??**. ZIF-8 uses the finite-difference Hessian calculation described in Appendix **??**.



(a) MOF-5 (b) MOF-303 (c) ZIF-8

Figure 4: Empty framework starting structures rendered with Chemiscope from the project's structure dataset. MOF-5 contains 424 atoms; the MOF-303 and ZIF-8 $2 \times 2 \times 2$ supercells contain 1024 and 2208 atoms, respectively. Cell outlines indicate the periodic simulation boxes. Views are independently scaled to reveal the pore geometry.

3.2 Structure preparation and initialization

The MOF-5 calculations use `input/mof5.pdb` for the host and `input/ch4.gro` for the methane template. The preparation routine reads the periodic host and single-molecule template. Randomly oriented guest molecules are placed at fractional positions in the cell, rejecting overlaps with the host or previously placed guests under periodic boundary conditions. The current routine uses a minimum intermolecular atom–atom separation of 1.5 Å and up to 20000 placement

attempts per molecule. If sequential insertion stalls, automatic repacking allows up to 20 rounds, removing a random quarter of the inserted molecules between rounds and restarting the full packing every fourth round. Guest geometry is held fixed only during insertion. The original MOF-5/methane structure archives do not retain the insertion-budget metadata, so these current preparation settings cannot be verified retrospectively for those structures.

The campaign seed convention is $20250000+100T+r$, where T is the numerical temperature in kelvin and r is the replica index. The reported runs use $r = 1$, giving velocity seeds from 20267501 at 175 K to 20292501 at 425 K. Prepared structures are shared between PET-MAD and PET-SOL at each loading and temperature. Velocities are drawn from a Gaussian distribution at the target temperature, with initial total linear and angular momentum removed. Each temperature is initialized separately.

The archived inputs for the 66 reported MD trajectories contain no pre-MD minimization command. The current preparation workflow adds a fixed-cell LAMMPS FIRE relaxation before assigning velocities, with a maximum Cartesian force-component tolerance of 0.05 eV/Å, zero energy-change tolerance, limits of 2000 iterations and 20000 force evaluations, and maximum displacement 0.05 Å. Failure to meet the force criterion stops a fresh run; the MD timestep is restored after minimization. These settings describe the updated initialization procedure, distinct from the archived MD inputs and from the post-MD relaxations used to obtain Hessians.

3.3 PET-MAD and PET-SOL

Two pretrained potentials are used: PET-MAD (`pet-mad-1.5-s-40nn`) and PET-SOL (`pet-sol-s-best`). Their local PET-JAX metadata records the common architecture in Table 1; their weights differ. The architecture and its operations are described in Section 2.6.

PET-MAD-1.5 was obtained by fine-tuning a PET model pretrained on OMat24 on the chemically diverse MAD-1.5 dataset, spanning 102 elements and including molecules, clusters, crystals, and low-dimensional structures. Its reference energies, forces, and periodic stresses were computed with all-electron r^{2}SCAN DFT. Published training used a weighted loss on these targets, AdamW optimization, learning-rate warm-up, and cosine annealing [29]. This describes the model family; the architecture of the local 40-neighbor checkpoint is recorded below.

For PET-SOL, the archived checkpoint records full-model fine-tuning from a pre-existing checkpoint, with mean-squared-error losses on energies and coordinate gradients (forces), weighted 10 and 1, respectively. Its saved settings specify a batch size of 32, learning rate 10^{-5}, and a maximum of 50 epochs [30]. The archive does not identify the training dataset or DFT functional. Both pretrained models are used here without further fitting to the MOF heat-capacity results.

Table 1: Architecture settings of PET-MAD and PET-SOL

Setting	Value
R_c, Δ_{R_c}	8.0 Å, 0.5 Å
n_{GNN}, n_{TL}	3, 1
d_{PET}	256
Attention heads	8
Transformer feed-forward width	512
Node feature width, `d_node`	1024

LAMMPS evaluates the exported metatomic model on a GPU for MD, while PET-JAX/SADMOF

evaluates the corresponding coordinate Hessians. The PET-MAD checkpoint and export have basename `pet-mad-1.5-s_40nn_nostress`, with the PET-JAX conversion `pet-mad-1.5-s_40nn_jax`; the PET-SOL basename is `pet_sol-s-best_nostress`, with `_jax` appended for the conversion. The exports are generated with `upet.save_upet`; conversion preserves the pretrained weights and does not retrain the model. The local PyTorch implementation uses `metatrain` 2026.3.1, and setup pins PET-JAX to commit `ed0add4`.

The generated configurations mark the models as stress-validated. Quantitative use nevertheless requires energy, force, and stress checks on representative loaded structures, together with consistent energy scaling and adaptive-cutoff derivative conventions across the two implementations (Appendix **??**). A configuration flag alone does not establish agreement. All terms of each hybrid curve use its selected model; the two potentials are analyzed separately.

3.4 NPT simulation

The archived LAMMPS inputs use `units metal` and `atom_style atomic`. A Nosé–Hoover-chain thermostat and MTTK barostat propagate a fully flexible triclinic cell. The `tri` pressure setting targets 1 bar for each normal component and zero shear stress, allowing both cell lengths and tilts to vary. Table 2 lists the common parameters for all 66 trajectories.

Table 2: Parameters shared by the reported MOF-5/methane NPT runs. The discarded interval is an analysis choice, not proof of equilibration.

Parameter	Value
T, ΔT	175–425 K, 25 K
P	1 bar
τ_T, τ_P	100, 1000 fs
t, Δt	500 ps, 0.5 fs
t_{eq}	100 ps
Frame interval	0.05 ps

Thermodynamic logs contain potential and kinetic energies, temperature, volume, the pressure tensor, and cell dimensions. Trajectory records contain atom identities, unwrapped positions, velocities, and forces. Analysis retains steps 200000–1000000, inclusive, corresponding to 8001 samples at the recorded output interval. Logs and configurations are matched by timestep. Restart continuations restore the saved state and run to the original final-step target without assigning new velocities; they form part of the same trajectory.

The enthalpy analysis follows Section 2.4, using Eq. (2.41) with $\Delta T = 25\,\mathrm{K}$, so each derivative uses two trajectories separated by 50 K. The extra 175 K and 425 K runs support centered derivatives at the endpoints of the reported 200–400 K interval. Energy, volume, temperature, and structural diagnostics are examined over the retained segment; the corresponding convergence checks are reported in Appendix **??**.

3.5 Hybrid C_P

3.5.1 Hybrid estimate used in this work

We use the hybrid approximation of Kapil *et al.* [21] to combine the classical anharmonic response sampled by NPT dynamics with the quantum vibrational populations described in Section 2.4.

Classical MD includes thermal motion and host–guest interactions beyond a quadratic potential, but assigns classical populations to nuclear vibrations. A purely harmonic quantum calculation accounts for quantized vibrations around one minimum but omits the anharmonic motion sampled along the trajectory.

The approximation starts from the full classical MD heat capacity, subtracts its harmonic reference, and adds the quantum harmonic value:

$$C_P^{\mathrm{hyb}}(T) \simeq C_{P,\mathrm{cl}}^{\mathrm{anh}}(T) + C_{V,\mathrm{qn}}^{\mathrm{har}}(T) - C_{V,\mathrm{cl}}^{\mathrm{har}}(T). \tag{3.1}$$

Here $C_{P,\mathrm{cl}}^{\mathrm{anh}}$ is the full classical heat capacity from the temperature derivative of mean enthalpy, not only its anharmonic part. The two harmonic terms use the same retained modes of the complete loaded system. Their difference replaces the classical vibrational contribution by its quantum counterpart without counting it twice; it tends to zero in the classical high-temperature limit. Using the loaded Hessian includes guest vibrations and their coupling to the framework.

In this work, Eq. (3.1) produces the hybrid curves used to compare models, guest loadings, and experimental references. Dividing by the total framework-plus-guest mass gives the reported J/g/K. The empty-framework harmonic curve remains a separate reference and is not the correction applied to a loaded system.

This construction retains classical anharmonicity while approximating the quantum correction harmonically. It neglects quantum anharmonic effects and assumes that the quantum change in $C_P - C_V$ is small. Reusing a single fixed-cell spectrum also neglects temperature-dependent changes in the reference frequencies and the distribution of occupied minima. The hybrid curves are therefore approximate estimates, whose sampling and reference stability must be checked.

3.5.2 Harmonic references and numerical assembly

For each model and loading, the final configuration of the 400 K trajectory is relaxed with ASE FIRE using the matching metatomic potential. All atomic coordinates are optimized while the final MD cell is held fixed. The resulting single structure supplies the loaded Hessian used at every reporting temperature. The empty MOF-5 input is relaxed separately at its original cell with each model to provide a harmonic reference; no empty-MOF MD trajectory is used.

Table **??** records the force targets and completed optimization steps for the MOF-5/methane and empty MOF-5 references. The stopping criterion is the largest atomic force norm, which differs from the Cartesian-component criterion used in the current pre-MD initialization. The four unconverged methane-loaded relaxations reached their recorded step limits. The archives for converged runs retain the completed step count but not the requested maximum budget; these values should therefore not be interpreted as iteration caps. FIRE uses the ASE defaults for its remaining optimizer controls, which are not separately recorded in the relaxation metadata.

PET-JAX/SADMOF computes the MOF-5 and MOF-303 Hessians with the sparse AD parameters in Table 3. The ZIF-8 results instead use the finite-difference calculation described in Appendix **??**. The normal-mode construction and quantum harmonic heat capacity are derived in Section 2.4.3, and the sparse AD algorithm is described in Appendix **??**. The sparse AD calculations symmetrize the Hessian, apply the force-constant row-sum acoustic treatment, and diagonalize the mass-weighted matrix while preserving the sign of unstable eigenvalues.

Strict assembly rejects unexpected imaginary modes and more than three near-zero modes.

Table 3: Hessian and heat-capacity settings for the MOF-5/methane analysis. The archived Hessians use float64, overriding the float32 TOML template.

Parameter	Value
k	3
Near-zero threshold	$\lvert\widetilde{\nu}\rvert \leq 1\,\mathrm{cm}^{-1}$
Imaginary-mode threshold	$\widetilde{\nu} < -1\,\mathrm{cm}^{-1}$

The reported exploratory curves instead retain only frequencies above $1\,\mathrm{cm}^{-1}$, using the same mask for the quantum and classical harmonic terms. This choice permits evaluation of the correction but does not establish stability of the reference structures; the corresponding diagnostics are reported in Appendix **??**.

Assembly uses Eq. (3.1) on the reporting grid in Table 3. The empty-framework reference uses the host mass for normalization. Sampling and model uncertainties are treated in the following subsection.

3.6 Uncertainty quantification

3.6.1 LLPR and CEA

When using a machine learning model to predict quantities, there are many sources of uncertainty. The model may not have been trained on data that is representative of the system being studied. It may not capture all relevant physics, which depends on the first principles methods used when doing DFT calculations. If we wanted to have better uncertainty estimates, we would need to have the model trained on different datasets and use these models to predict the same quantity. We could then have a better idea on the uncertainty of the model predictions. Additionaly, we could also build complete different models with different architectures and train them with different datasets. Combining the results from each model to have a final prediction and uncertainty estimate is what we call ensemble learning. In traditional machine learning, ensemble learning is a technique that combines the predictions of multiple models to improve overall performance and provide more robust uncertainty estimates. In practice, ensemble learning is widely used in various machine learning tasks, including regression, classification, and forecasting. One of the most efficient models are tree-based models, such as Random Forests and Gradient Boosting Machines, which combine the predictions of multiple decision trees to improve accuracy and reduce overfitting. However, training multiple models, especially in the case of MLIPs, is an expensive task. Instead of training multiple models, we can use the last-layer prediction rigidity (LLPR) [38] method to estimate the uncertainty of a single model. The idea is that instead of training multiple models, we take the model we are using for the simulation and we create new models by just changing its last layer. The models with the different last layers could be considered as different models that were trained on different datasets. This method is cheap and efficient, and it has been shown to provide good uncertainty estimates for MLIPs. For central energy $\bar{U} = \boldsymbol{w}_0^{\mathrm{T}} \boldsymbol{f}(\boldsymbol{x})$ andcovariance-set features $\boldsymbol{F}$,

$$\begin{aligned} \sigma^2_{\mathrm{LLPR}}(\boldsymbol{x}) = \alpha^2 \boldsymbol{f}(\boldsymbol{x})^{\mathrm{T}} \\ \times \left(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{F} + \lambda \boldsymbol{I}\right)^{-1} \boldsymbol{f}(\boldsymbol{x}). \end{aligned} \tag{3.2}$$

The regularizer λ stabilizes weakly constrained directions; α is calibrated against energy residuals on a disjoint reference-labeled set using the base potential's electronic-structure definition. Predictions cannot calibrate their own error.

The exported `energy_uncertainty` is an energy standard deviation. Property propagation instead needs persistent, signed energy variations: Each new model is generated by sampling the last-layer weight vector $\boldsymbol{w}_i$ from the following distribution

$$\boldsymbol{w}_i \sim \mathcal{N}\left[\boldsymbol{w}_0, \alpha^2 \left(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{F} + \lambda \boldsymbol{I}\right)^{-1}\right], \tag{3.3}$$

The model is then created by multiplying the sampled weight vector with the weights from all the previous layers of the model, which are kept frozen.

$$U^{(i)}(\boldsymbol{x}) = \boldsymbol{w}_i^{\mathrm{T}} \boldsymbol{f}(\boldsymbol{x}) \tag{3.4}$$

The advantage of this method is that it is not only cheap to generate the models, but also we do not need to rerun the simulations with them. The model used to run the simulation can be considered to be the central model and the mean of all the other models generated by sampling the last layer is equal to the central model. That means that we can use the same trajectory generated by the central model to evaluate the quantities with the other models, which will be treated as uncertainties. We can then compute the difference between the potential of the members and the central model as well as the enthalpy of each member.

$$\Delta U_t^{(i)}(T) = U_t^{(i)}(T) - \bar{U}_t(T), \tag{3.5}$$

$$H_t^{(i)}(T) = K_t + U_t^{(i)}(T) + PV_t, \tag{3.6}$$

In particular, if we want to calculate the ensemble average of the enthalpy as a function of time, we can use the following equation

$$\left\langle H^{(i)}(T) \right\rangle = \frac{\sum_t e^{-\beta \Delta U_t^{(i)}(T)} H_t^{(i)}}{\sum_t e^{-\beta \Delta U_t^{(i)}(T)}}, \tag{3.7}$$

It is done using the Boltzmann factors of the energy differences between the central model and the members. When the energy differences $\Delta U_t^{(i)}$ are small, the direct reweighting can be approximated by a first-order cumulant expansion approximation (CEA):

$$\begin{aligned} \left\langle H^{(i)}(T) \right\rangle \approx\ & \left\langle H^{(i)}(T) \right\rangle_{\bar{U}} - \beta \operatorname{cov}_{\bar{U}} \left(H^{(i)}(T), \Delta U^{(i)}(T)\right), \\ & \beta = (k_{\mathrm{B}} T)^{-1}. \end{aligned} \tag{3.8}$$

The CEA is a useful approximation when the energy differences are small, but it is not exact. Direct reweighting remains a diagnostic: $N_{\mathrm{eff}} = 1/\sum_t \widetilde{w}_t^2$ should remain adequate, and $\mathrm{var}_t(\beta \Delta U_t^{(i)}) \ll 1$ is needed for controlled first-order CEA. Applying Eq. (2.41) to each member's CEA mean enthalpies gives $C_{P,\mathrm{cl}}^{(i)}(T)$.

We can then simply calculate the uncertainty at each temperature as the standard deviation of the heat capacities of all the members:

$$\sigma_{C_{P,\mathrm{cl,anh}}}(T) = \left\{ \frac{1}{M-1} \sum_{i=1}^{M} \left[C_{P,\mathrm{cl}}^{(i)}(T) - \overline{C}_{P,\mathrm{cl}}(T) \right]^2 \right\}^{1/2}. \tag{3.9}$$

This term corresponds to the classical anharmonic heat capacity uncertainty. To compute the uncertainty of the quantum harmonic heat capacity, we need first to select the last frame in the simulation, relax the structure and then compute the Hessian. We then calculate the eigenvalues of the Hessian matrix, which correspond to the vibrational frequencies of the system. We will compute the Hessian for each model and its eigenvalues, and then we will compute the quantum harmonic heat capacity for each model using Eq. (**??**). Finally, we can compute the uncertainty of the quantum harmonic heat capacity as the standard deviation of the heat capacities of all the models:

$$\sigma_{C_{V,\mathrm{qn,har}}}(T) = \left\{ \frac{1}{M-1} \sum_{i=1}^{M} \left[C_{V,\mathrm{har}}^{(i)}(T) - \overline{C}_{V,\mathrm{har}}(T) \right]^2 \right\}^{1/2}. \tag{3.10}$$

The total uncertainty of the hybrid heat capacity is then obtained by combining the uncertainties of the classical anharmonic and quantum harmonic heat capacities in quadrature:

$$\sigma_{C_P}(T) = \sqrt{\sigma^2_{C_{P,\mathrm{cl,anh}}}(T) + \sigma^2_{C_{V,\mathrm{qn,har}}}(T)}. \tag{3.11}$$

4 Results

4.1 MOF-5

Open squares in the MOF-5 plots show the supplied experimental data for empty MOF-5 powders from Ming *et al.* [23]. These mass-specific measurements provide a zero-loading reference and are shown only at the supplied temperatures, without extrapolation.

4.1.1 CH_4

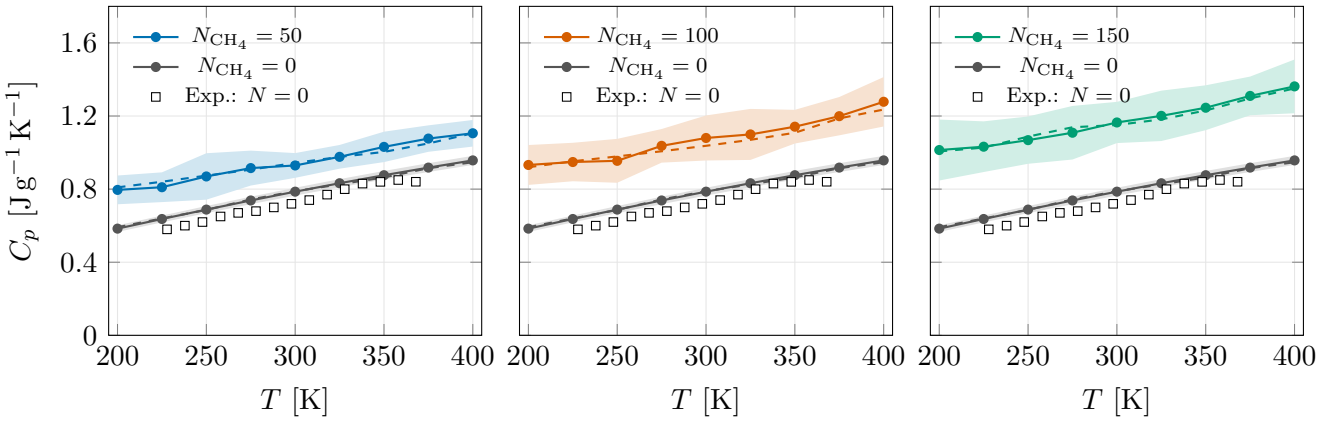


Figure 5: MOF-5 C_P with 50, 100, and 150 CH_4 molecules per cell using PET-MAD. The dashed lines are from Kapil *et al.* [21]. Open squares are experimental empty-MOF-5 data from Ming *et al.* [23]. Shading denotes standard uncertainty; details are in Appendix **??**.

The potential used in **??** [21] is called QuickFF, which was specially designed for MOFs and trained on DFT data. We can see that using PET-MAD gives very similar results to the QuickFF potential, with differences of at most 0.043 J/g/K at 300 K. In other words, PET-MAD, which is a more general-purpose potential, is able to reproduce the results of a specialized potential for MOFs. If it is faster and more efficient than QuickFF, it could be a better choice for simulating other MOFs in the future. Especially if we want to simulate larger MOFs or use larger supercells, PET could be a better choice because it is more general and can be used for other systems as well. What is even more surprising is that the reference data was obtained using PIMD, which is significantly more expensive than classical MD. But even using classical MD with PET-MAD, we are able to reproduce very close results to waht we get with PIMD with QuickFF. Thus by using PET-MAD, classical MD, and the hybrid method, we can obtain very accurate results for the heat capacity of MOFs with a much lower computational cost than using PIMD with QuickFF.

The major difference between our results and those of Kapil *et al.* is how the uncertainty is calculated. There is an uncertainty with the MD simulation itself, but on top of that, there is also an uncertainty with the model itself. Using 64 committee members yields the uncertainty bands shown in Figure 5. Because the uncertainty bands would be overlapping, we split the three loadings into separate panels for clarity. Something that stands out is the fact that the uncertainty bands for the empty framework are much smaller than those for the loaded systems. This is because no MD simulation was required to compute its heat capacity, the uncertainty is only given by 3.11, which is much smaller than the uncertainty of the MD simulation. Furthermore, the uncertainty bands also get larger as the loading increases. This could be explained by the

fact that for an empty framework, the atoms are already in a stable configuration, evaluating the energy of the system with the different committee members does not yield very different results. For the loaded systems, the guest molecules are not in a stable configuration. By changing the last layer of the model, the energy of the system is more sensitive to the guest molecules. The guest-guest and guest-host interactions are more sensitive to the model, which is why the uncertainty bands are larger for the loaded systems.

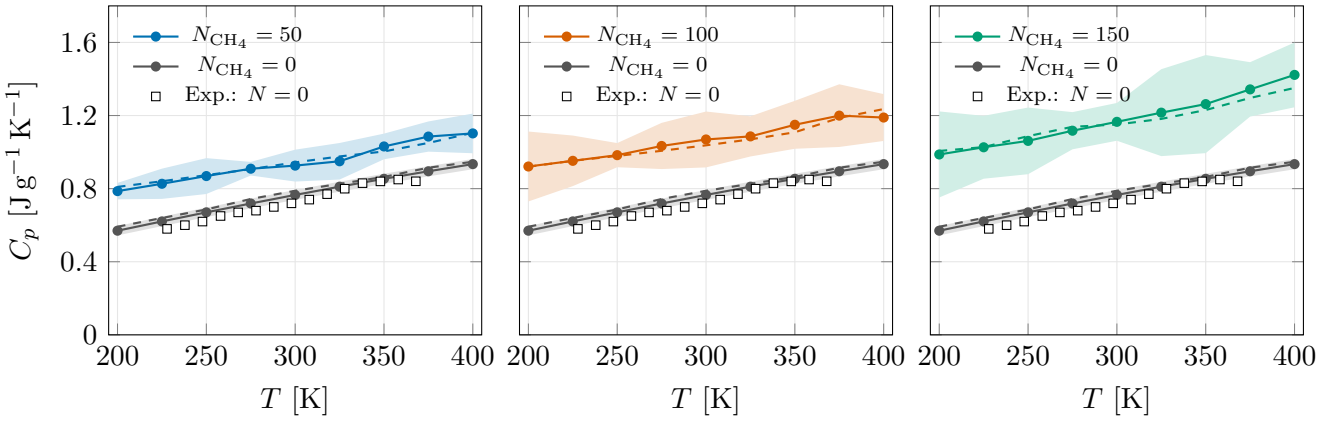


Figure 6: MOF-5 C_P with 50, 100, and 150 CH_4 molecules per cell using PET-SOL. The dashed lines are from Kapil *et al.* [21]. Open squares are experimental empty-MOF-5 data from Ming *et al.* [23]. Shading denotes standard uncertainty; details are in Appendix **??**.

Here, we can arrive at a similar conclusion as with PET-MAD, which is that PET-SOL reproduces accurately the results of QuickFF, with differences of at most 0.043 J/g/K at 300 K. PET-SOL was specifically trained on solvents, but it offers no advantage over PET-MAD, which was trained on a more general dataset. The error obtained with PET-SOL is actually slightly larger than the one obtained with PET-MAD. After having validated that both PET-MAD and PET-SOL are able to reproduce the results of QuickFF, we can carry out simulations using classical MD and the hybrid method to obtain the heat capacity of other MOFs and different guest molecules. To save computational resources, we will only use PET-MAD for the rest of the simulations, since it is more general and has showed smaller errors than PET-SOL.

4.1.2 CO_2 and H_2O

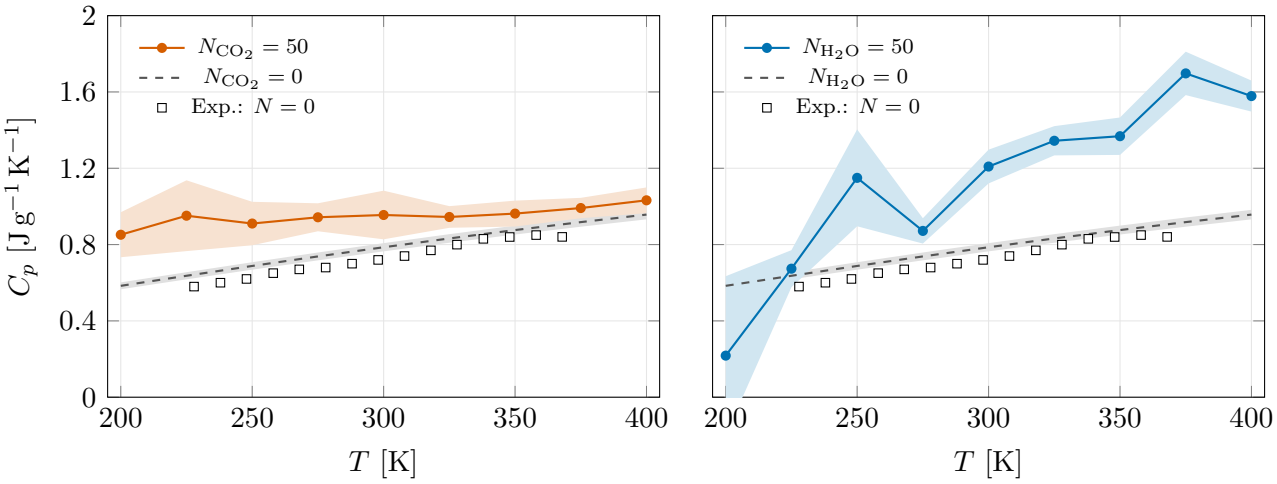


Figure 7: MOF-5 heat capacities with 50 CO_2 molecules (left) and 50 H_2O molecules (right) per cell using PET-MAD. Each panel shows the total hybrid c_P and the empty-framework harmonic c_V reference. Loaded curves use the host-plus-guest mass; empty references use the host mass. Open squares show the experimental empty-MOF-5 data of Ming *et al.* [23]. Shading denotes standard uncertainty; details are in Appendix **??**. Both y-axes start at zero; uncertainty bands extending below zero are clipped at the axis boundary.

No experimental data is available for the heat capacity of MOF-5 with CO_2 molecules and this calculation was not carried out by [21]. However, we can see that the qualitative behavior of the heat capacity curve is similar to that of CH_4. The only difference being that the curve seems to plateau and at 400 K, the heat capacity is very close to that of the empty framework. If we kept increasing the temperature, the empty framework heat capacity would eventually be larger than the loaded system, which is not physically correct. So for CO_2 loading, we need to be careful when interpreting the results at high temperatures. Something that was done in [21] is running multiple MD simulations for each temperature. The results were then averaged to obtain a more accurate heat capacity curve. Due to a lack of time for this project, only one MD simulation was run for each temperature, so there is still room for improvement.

Here, for 200 K, the heat capacity of the 50 H_2O loading is below the empty framework reference, which is contrary to what we would expect. Since the heat capacity should increase with the number of molecules in the system, this could indicate that something was wrong with the simulation at lower temperatures. Furthermore, we should keep in mind that in the temperature range of 200–400 K, the H_2O should be in three different phases: solid, liquid, and gas. In particular, between 250 and 275 K, a phase transition should occur between solid and liquid. In the plot, the heat capacity decreases significantly. This could indicate a discontinuity in the enthalpy, which is a signature of a phase transition. While usually, we would expect the heat capacity to increase at each temperature step. The same comments can be made for temperatures between 350 and 375 K, where the water molecules should undergo another phase transition between liquid and gas. There is a slightly smaller discontinuity in the heat capacity, which could indicate a phase transition. However, capturing phase transitions with MD simulations is notoriously difficult and we should be careful when interpreting these results. In particular, further analysis is required to determine whether PET is capable of capturing phase transitions.

We would need experimental data to compare with our results, but not much experimental data is available for water in MOF-5.

4.2 MOF-303

These PET-MAD calculations use a $2 \times 2 \times 2$ supercell containing 50 CH_4, 50 CO_2, or 50 H_2O molecules. The guest-specific plots below show total heat capacities alongside the empty-framework reference and relevant experimental measurements [22].

4.2.1 CH_4, CO_2, and H_2O

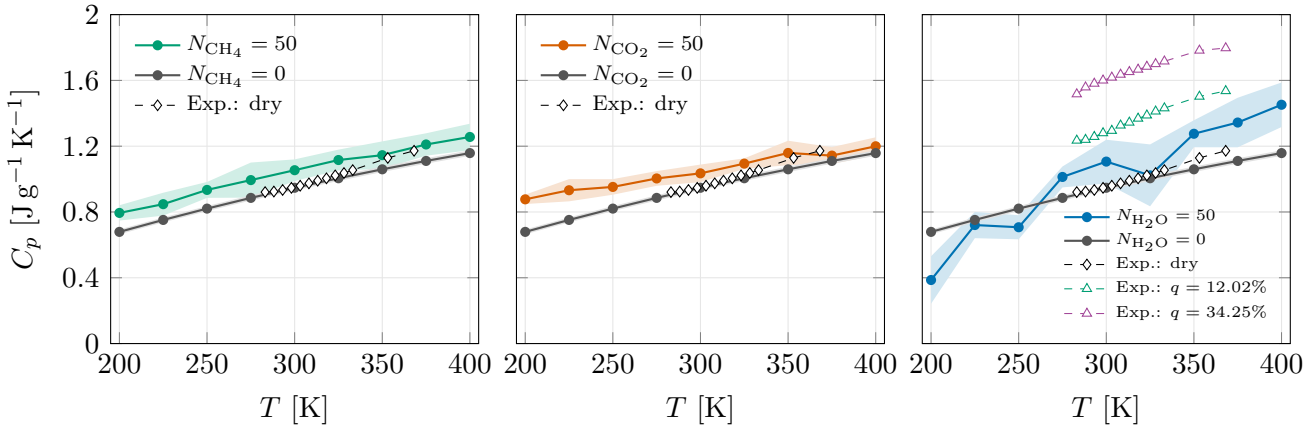


Figure 8: PET-MAD heat capacities for MOF-303 with 50 CH_4 (left), 50 CO_2 (middle), and 50 H_2O (right) molecules per $2 \times 2 \times 2$ supercell. Each panel shows the total hybrid c_P and empty-framework harmonic c_V reference. Loaded curves use the host-plus-guest mass; empty references use the host mass. The methane panel uses its campaign-specific empty Hessian. Shading denotes standard uncertainty. Dashed experimental curves connect dry-MOF-303 measurements in each panel and water loadings $q = 12.02\%$ and 34.25% in the right panel [22]; experimental uncertainties were not supplied. Here q is the fraction of saturated water loading, and the simulation corresponds to $q = 16.76\%$. Dry measurements refer to the empty framework. All y-axes start at zero.

The experimental loading in [22] is reported as the ratio of the saturated water loading, $q = m_{\text{water}}/m_{\text{water,sat}}$. The reported dry and saturated sample masses, 7.3990 and 10.5358 mg, give a saturated water-to-dry-framework mass ratio of 0.42395 [22]. The 50 water molecules in the present 1024-atom host give a water-to-host mass ratio of 0.07106 and hence $q = 0.16761$. This composition lies between the measured $q = 0.1202$ and $q = 0.3425$ samples. However, our results lie below both of them, while it should be between them. It is hard to accurately reproduce the experimental results, because we cannot be sure of the exact loading of water in the MOF-303 sample. Whereas in the simulations, we select an exact number of guest molecules inside the MOF. Once again, the heat capacity with loaded H_2O is below the empty-framework at lower temperatures. The curve is overall not strictly monotonic, and the low-temperature behavior is not physically reasonable. For H_2O loaded MOFs, it might be better idea to try to simulate it with PIMD, which is more expensive but could give more accurate results. Testing with another potential might give better results. In any case, we should be careful when interpreting the results for H_2O loaded MOFs, especially at low temperatures.

4.3 ZIF-8

The PET-MAD calculations contain 50 CH_4 or 54 CO_2 molecules in a 2×2×2 supercell, containing 96 $Zn(MeIm)_2$ formula units. The CO_2 molar loading is therefore $X = 54/96 = 0.5625$, close to the experimental $X = 0.56$ sample of Rosen *et al.* [25].

4.3.1 CH_4 and CO_2

For CO_2, at 300 K, $c_{cl} = 2.426$ and $\Delta c_{har} = -1.460$ give $c_{hyb} = 0.967 \pm 0.151\,J\,g^{-1}\,K^{-1}$. The hybrid curve rises from 0.805 at 200 K to 1.200 J/g/K at 400 K (Figure 9). Different guest concentrations and mass normalizations must be considered when comparing these values with the other frameworks.

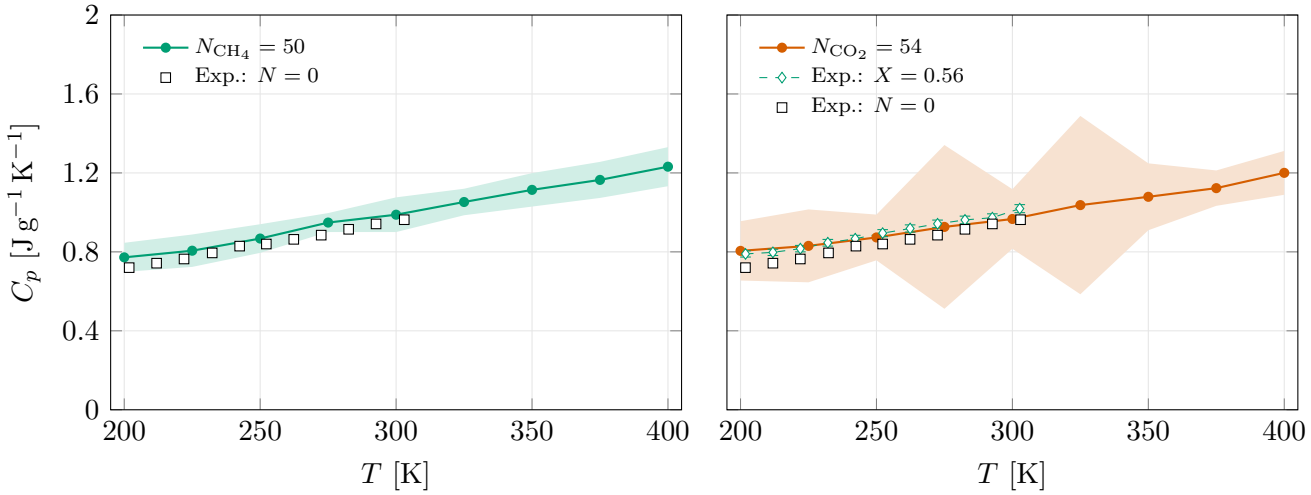


Figure 9: PET-MAD total hybrid heat capacities with standard uncertainty for 50 CH_4 (left) and 54 CO_2 (right) molecules in the ZIF-8 2 × 2 × 2 supercell. The CO_2 panel includes experimental measurements at $X = 0.56$ mol CO_2/mol ZIF-8 [25]; the simulation has $X = 0.5625$. The experimental molar values from Supporting Information Table 3 are divided by the loaded sample molar mass, 252.3 g/mol, to give the same total-sample mass basis as the simulation. Experimental error bars show 2% standard uncertainty in this temperature range; dashed lines connect measured points. The experiment reports sealed-cell C, whereas the simulation estimates C_P. Open squares show independent experimental empty-ZIF-8 (SOD) data [24], converted from molar units using 227.576 g/mol. The new digitized empty-framework datasets do not include uncertainty values, so no error bars are inferred for them. Both y-axes start at zero.

On the common 200–300 K grid, linear interpolation between neighboring experimental temperatures gives signed simulation differences ranging from approximately +2.2% to −4.0%. At 300 K, the interpolated experimental value is approximately 1.007 J/g/K, compared with the simulated $0.967 \pm 0.151\,J\,g^{-1}\,K^{-1}$. The 2% experimental standard uncertainty corresponds to approximately 0.020 J/g/K at this temperature and does not include a separate interpolation uncertainty.

This numerical agreement is qualified by the measurement conditions. The experiment fixes the total CO_2 amount in a sealed pan; neglecting pan expansion, the authors describe its heat capacity as approximately constant-volume for the entire system, including any free CO_2 outside the crystal [25]. The periodic NPT simulation instead contains all 54 CO_2 molecules inside the framework. The initial experimental sealing pressure is not a pressure maintained throughout the

temperature scan. In addition, the existing 200–400 K simulation curve samples only the upper tail of the reported 70–220 K structural anomaly. We see that our results are surprisingly close to the experimental measurements.

5 Conclusion

We saw that the PET architecture is able to reproduce the results of a specialized potential for MOFs like QuickFF. As a result, we can use more general purpose potentials like PET-MAD to simulate the heat capacity of MOFs with different guest molecules. For some cases, like H_2O loaded MOFs, the results are not physically reasonable at low temperatures. We may need to further validate the results with PIMD simulations or other potentials to ensure that the results are accurate. Given that carrying out experiments for measuring the heat capacity of MOFs is complicated, using MD simulations and the hybrid method is a good alternative to obtain the heat capacity of MOFs with different guest molecules. PET-MAD and PET-SOL give exploratory room-temperature heat capacities rising from about 0.93 to 1.16 J/g/K with methane loading. Harmonic quantization removes 59–63% of the classical term; guest mobility supports retaining anharmonic MD. Broad agreement with digitized reference trends does not establish quantitative accuracy. Additional PET-MAD estimates at 300 K are 0.955/1.209 for CO_2/H_2O in MOF-5, 1.035/1.106 for CO_2/H_2O in MOF-303, and 0.967 J/g/K for CO_2 in ZIF-8. These values use different guest concentrations and do not establish a ranking of hosts. Experimental overlays place the MOF-303/water estimate below the measured neighboring loading curves at 300 K, while the ZIF-8/CO_2 estimate is within about 4% of the matched-composition measurement. Loading, mass-normalization, and sealed-cell versus constant-pressure conditions qualify these comparisons. Water-loaded trajectories show particularly strong drift, and all loaded spectra contain imaginary modes. Stable spectra, independent trajectory/minimum sampling, and validated paired uncertainty propagation are needed before interpreting these curves as converged predictions.

A Transformers and attention

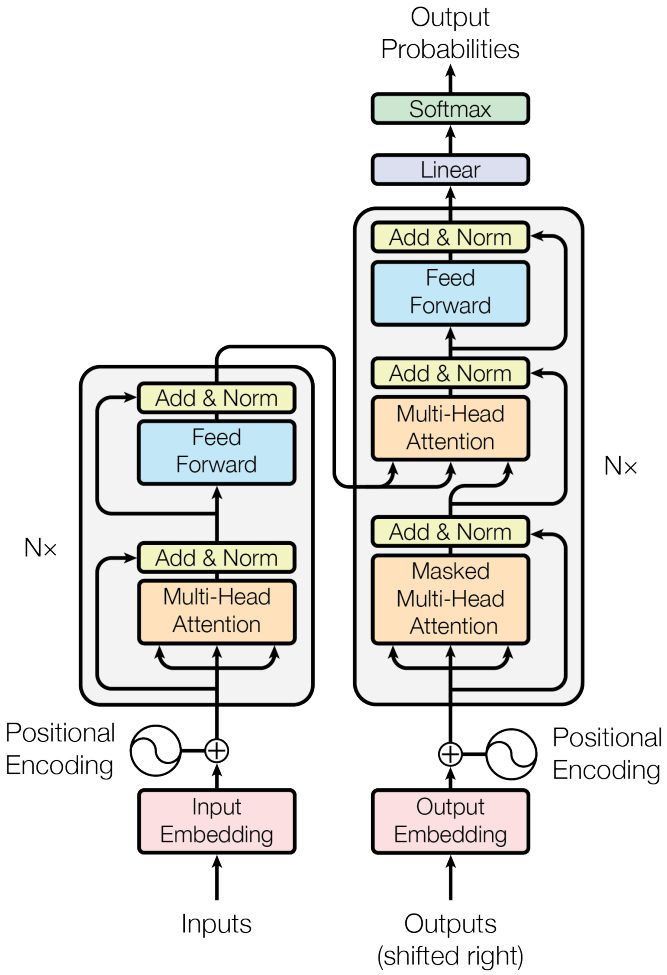


Figure 10: Transformer architecture developed in Ref. [26]. The attention mechanism is the part in gray. The part of the transformer on the left is called the encoder, and the part on the right is called the decoder. The encoder takes a sequence of tokens as input and produces a set of vectors, whose representations capture the meaning of the input tokens and their context. The decoder uses these vectors to generate an output sequence and guesses the next token in the sequence.

The transformer architecture has been revolutionary in the field of AI as a whole. It was initially developed for natural language processing (NLP) tasks, in particular for translation. But it has since been applied to a wide range of other domains, including computer vision, speech processing, code generation, protein folding. It is a state-of-the-art architecture that has been used to build some of the most powerful AI models, such as GPT-3 and GPT-4. It has also been applied to material science, in particular in MLIPs, where it has been used to predict the properties of materials and molecules.

A.1 Tokenization and embedding

The original transformer architecture [26] was designed for sentence translation. But it was then used to other NLP tasks, such as text classification and question answering. Therefore, the transformer architecture takes a text as input. However, a text is a sequence of strings, which is not numerical data and can't be directly processed by a neural network. To convert a sentence into a numerical representation, each word (or punctuation mark) in the sentence is first separated

into a token, and then each token is mapped to a vector representation, called an embedding. Each token embedding is a vector living in a high-dimensional space, and the embedding vectors are stacked together to form a matrix $\boldsymbol{X}$. The way to interpret these vectors is that each token is represented as a point in a high-dimensional space, and the distance between two points reflects the semantic similarity between the corresponding tokens. For instance, the tokens "cat" and "dog" are both animals, so their embeddings should be close to each other in the embedding space. In addition, the transformer architecture also uses positional encoding to encode the order of the tokens in the sequence. This is especially important for words that could have different meanings depending on their position in the sentence. For instance, the word "bank" could refer to a financial institution or the side of a river, and its meaning can be inferred from the surrounding words. So the embedding of the word "bank" could have different values depending on its position in the sentence, and the positional encoding helps to capture this information. These embeddings are something that has to be learned from the data, and they are usually initialized randomly and then updated during training. In the case of the PET architecture, the token embeddings are already explained in Section 2.6. So what is relevant for us is to understand the next step in the transformer architecture, which is the attention mechanism.

A.2 Attention mechanism

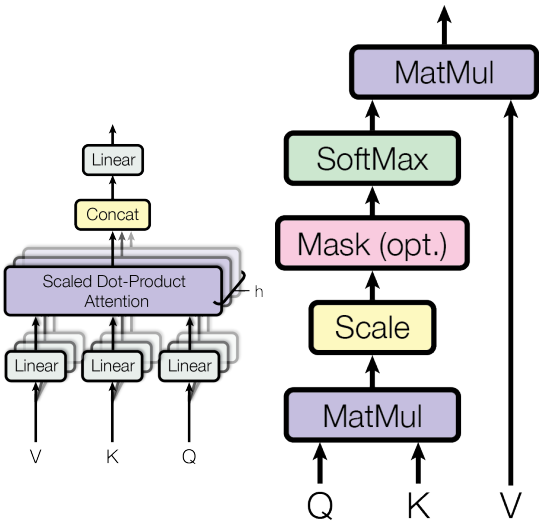


Figure 11: On the left, we have a multi-head attention mechanism, which is the orange part of the transformer architecture in Figure 10. On the right, we see the scaled dot-product attention mechanism, which is the blue part on the left figure. Pictures are taken from Ref. [26].

With our input matrix $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, where n is the number of tokens and d is the embedding dimension, we compute the queries, keys, and values as follows:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V, \tag{A.1}$$

with $\boldsymbol{W}_Q, \boldsymbol{W}_K \in \mathbb{R}^{d \times d_k}$ and $\boldsymbol{W}_V \in \mathbb{R}^{d \times d_v}$. Thus $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n \times d}$. These matrices are what the attention mechanism uses to compute the attention matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, which contains the attention weights for each token pair.

$$\boldsymbol{A} = \mathrm{softmax}_{\mathrm{row}}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}}{\sqrt{d_k}} + \boldsymbol{M}\right), \tag{A.2}$$

To better understand what $\boldsymbol{A}$ represents, we can look at its coefficients. Suppose we have a sentence "The animal didn't cross the street because it was too tired". The word "it" refers to "the animal" and not "the street", so the attention weight between "it" and "the animal" should be high, while the attention weight between "it" and "the street" should be low. The term $\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}$ computes the dot product between the queries and keys, which measures the similarity between each token's query and every other token's key. The scaling factor $1/\sqrt{d_k}$ is used to prevent the dot products from growing too large. The $\boldsymbol{M}$ matrix is a mask that can be used to prevent certain tokens from attending to each other. For instance, if we are training the model to predict the next word in a sentence, we don't want the model to know the future words, otherwise it would defeat the purpose of predicting the next word, so we "mask" these words by setting their attention weights to zero. The softmax function is given by

$$\mathrm{softmax}_{\mathrm{row}}(\boldsymbol{Z})_{ab} = \frac{\exp(Z_{ab})}{\sum_{c=1}^{n}\exp(Z_{ac})}, \qquad a, b = 1, \ldots, n, \tag{A.3}$$

The softmax function normalizes the attention weights so that they sum to one across each row, which means that for each token, the attention weights represent a probability distribution over all other tokens. We add large negative values to the masked positions in $\boldsymbol{M}$ so that their softmax output is effectively zero. After computing $\boldsymbol{A}$, we can compute the output of the attention mechanism as a weighted sum of the values:

$$\boldsymbol{Y} = \boldsymbol{A}\boldsymbol{V}, \tag{A.4}$$

where $\boldsymbol{Y} \in \mathbb{R}^{n \times d_v}$ is the output of the attention mechanism. Each of the W_Q, W_K, and W_V are weight matrices that are fine-tuned during training. In the PET architecture, masking is not needed because there is no sequential ordering of the tokens, and all tokens can attend to each other.

A.3 Multi-head attention

We can extend the attention mechanism by dividing the input tokens $\boldsymbol{X}$ into subsets $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_H$, such that $d_v = d/H$, and constructing attention matrices $\boldsymbol{A}_1, \ldots, \boldsymbol{A}_H$ for each subset, this is called multi-head attention. We can then concatenate the outputs of each head to form the final output:

$$\mathrm{MHA}(\boldsymbol{X}) = \mathrm{Concat}(\boldsymbol{Y}_1, \ldots, \boldsymbol{Y}_H)\boldsymbol{W}_O, \qquad \boldsymbol{W}_O \in \mathbb{R}^{Hd_v \times d}. \tag{A.5}$$

The two main advantages of using multi-head attention instead of simple attention are that it allows to make the process more parallelizable, and it allows the model to learn more complex relationships between the tokens, since it also adds more parameters to the model. A common choice is $d_k = d_v = d/H$ for $d = 256$ and eight heads this gives 32 components per head. These dimensions describe attention projections, rather than the width of a later energy readout head.

B Sparse automatic differentiation

Automatic sparse differentiation (ASD) is a method for efficiently computing derivatives of neural networks. It makes use of the sparsity of the Jacobian (or Hessian) matrix to reduce the memory usage of these matrices, by storing only the non-zero elements. The sparsity pattern of these matrices also allows us to compute the derivatives more efficiently, by avoiding unnecessary computations of zero elements. This method provides an exact computation of the derivatives, but we can make some approximations to further reduce the computations. The approximation is explained in the Hessian truncation section.

B.1 The Jacobian, JVPs, and VJPs

For a function $\boldsymbol{f} : \mathbb{R}^m \to \mathbb{R}^n$, the Jacobian matrix is defined as

$$\boldsymbol{J}(\boldsymbol{x}) = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_m} & \cdots & \frac{\partial f_n}{\partial x_m} \end{pmatrix} \in \mathbb{R}^{n \times m} \tag{B.1}$$

We can compute the directional derivative of $\boldsymbol{f}$ at $\boldsymbol{x}$ along a vector $\boldsymbol{v} \in \mathbb{R}^m$ as

$$\frac{\mathrm{d}}{\mathrm{d}\epsilon} \boldsymbol{f}(\boldsymbol{x} + \epsilon \boldsymbol{v}) \bigg|_{\epsilon=0} = \boldsymbol{J}(\boldsymbol{x})\boldsymbol{v}. \tag{B.2}$$

This is what we call a *Jacobian–vector product* (JVP). It is the instantaneous rate of change of $\boldsymbol{f}$ along the direction $\boldsymbol{v}$. Since the Jacobian at a point $\boldsymbol{x}$ is a linear map, the directional derivative is linear in $\boldsymbol{v}$. This means that we can recover all the columns of the Jacobian by evaluating the JVPs along the standard basis vectors of $\mathbb{R}^m$: $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_m$. Let's illustrate this with a simple example. Consider the function $\boldsymbol{f} : \mathbb{R}^4 \to \mathbb{R}^3$

$$\boldsymbol{f}(\boldsymbol{x}) = \begin{pmatrix} x_1^2 + x_2 \\ \sin(x_2) + x_3^2 \\ e^{x_4} \end{pmatrix}, \tag{B.3}$$

The Jacobian is

$$\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \\ 0 & \cos(x_2) & 2x_3 & 0 \\ 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{B.4}$$

If we try evaluating the JVPs along the standard basis vectors, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_1 = \begin{pmatrix} 2x_1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_2 = \begin{pmatrix} 1 \\ \cos(x_2) \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_3 = \begin{pmatrix} 0 \\ 2x_3 \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_4 = \begin{pmatrix} 0 \\ 0 \\ e^{x_4} \end{pmatrix}. \tag{B.5}$$

Which exactly recovers the columns of the Jacobian in Eq. (B.4). However, the same result could have been obtained if instead of evaluating the JVPs along the standard basis vectors of $\mathbb{R}^m$, we had used the standard basis vectors of $\mathbb{R}^n$ and evaluated the vector-Jacobian products

(VJP) along the standard basis vectors of $\mathbb{R}^n$.

$$\boldsymbol{e}_1^{\mathrm{T}}\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \end{pmatrix}, \quad \boldsymbol{e}_2^{\mathrm{T}}\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 0 & \cos(x_2) & 2x_3 & 0 \end{pmatrix}, \quad \boldsymbol{e}_3^{\mathrm{T}}\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{B.6}$$

This time, we recover the rows of the Jacobian instead of the columns. The advantage of using VJPs here instead of JVPs is that if $n \ll m$, which means we can recover the Jacobian with fewer evaluations. This is particularly useful when dealing with neural networks, where the number of outputs is usually much smaller than the number of inputs. In particular, in the case MLIPs, the number of outputs is only one, the energy of the system, while the number of inputs depends on the number of atoms in the system and can be much larger. For a composition of functions $\boldsymbol{f} = \boldsymbol{h} \circ \boldsymbol{g}$, the chain rule gives the Jacobian as

$$\boldsymbol{J}_f(\boldsymbol{x})\boldsymbol{v} = \boldsymbol{J}_h(\boldsymbol{g}(\boldsymbol{x}))\big[\boldsymbol{J}_g(\boldsymbol{x})\boldsymbol{v}\big]. \tag{B.7}$$

B.2 Recovering multiple columns with a single evaluation

In some cases, we can recover multiple columns of the Jacobian with a single VJP evaluation. For instance, if we evaluate the VJP along the vector $\boldsymbol{v} = \boldsymbol{e}_1 + \boldsymbol{e}_3 + \boldsymbol{e}_4$, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{v} = \begin{pmatrix} 2x_1 \\ 2x_3 \\ e^{x_4} \end{pmatrix}. \tag{B.8}$$

The elements of this vector are the only nonzero entries in the first, third, and fourth columns of the Jacobian. By exploiting the sparsity of the Jacobian, we can recover multiple columns with a single evaluation. This is possible because column 1 has zeros in rows 2 and 3, column 3 has zeros in rows 1 and 3, and column 4 has zeros in rows 1 and 2. In other words, the nonzero entries of these columns do not overlap in any row. The nonoverlapping property is what allows us to recover multiple columns with a single evaluation. The last remaining column can be recovered with a second evaluation along $\boldsymbol{e}_2$. Thus, we can recover the entire Jacobian with only two evaluations instead of four as previously done.

B.3 Graph coloring

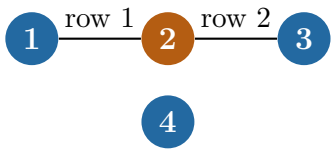
To better understand which columns can be grouped together, we can represent the sparsity pattern as a graph. Each column in $\boldsymbol{J}$ corresponds to a vertex in the graph and an edge connects two vertices if they have one or more nonzero entries in the same row. In our example, columns 1 and 2 have a nonzero entry in row 1, so they are connected by an edge. Same for columns 2 and 3, which have a nonzero entry in row 2. The set of vertices and edges forms a graph $G = (V, E)$, where $V = \{1, 2, 3, 4\}$ is the set of vertices (columns of $\boldsymbol{J}$) and $E = \{\{1, 2\}, \{2, 3\}\}$ is the set of edges (connections between columns). We can represent a graph with a diagram, where vertices are drawn as circles and edges as lines connecting them.

The next step is to assign a color to each vertex in the graph such that no two adjacent vertices share the same color. This process is called *graph coloring*. The goal is to find a coloring that uses the smallest number of colors possible $\chi(G)$, which is called the *chromatic number* of

(a) Sparsity pattern

$$\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \\ 0 & \cos(x_2) & 2x_3 & 0 \\ 0 & 0 & 0 & e^{x_4} \end{pmatrix}.$$

(b) Column conflict graph



Column 4 has no conflicts.

Group 1: $\{1, 3, 4\}$ Group 2: $\{2\}$

Figure 12: The same four-input example viewed as a pattern and as a graph. Colored coefficients mark possible nonzero entries. Graph edges record conflicts between columns, with the responsible row written beside each edge. Blue columns 1, 3, and 4 have no mutual conflicts and share one seed; orange column 2 uses another. A color denotes a group of columns.

the graph. The number of colors in the graph corresponds to the number of JVP evaluations needed to recover all columns of $\boldsymbol{J}$. The columns of the same color have no conflicts and can be computed together in a single JVP evaluation. In our example, $\chi(G) = 2$, as shown in Fig. 12. Columns 1, 3, and 4 can be assigned the same color (blue), while column 2 can be assigned another color (orange). As we saw previously, we could recover columns 1, 3, and 4 with a single JVP evaluation along $\boldsymbol{e}_1 + \boldsymbol{e}_3 + \boldsymbol{e}_4$, and column 2 with another JVP evaluation along $\boldsymbol{e}_2$. Which is consistent with the coloring we found. For this simple example, the chromatic number is easy to find by inspection. But in general, finding the chromatic number of a graph is an NP-hard problem. Therefore, we need to use certain algorithms to find a coloring that uses a small number of colors, even if it is not the minimum possible.

B.4 Decompression

After finding a coloring, we can evaluate the JVPs along the vectors belonging to each vector. We can group them in a matrix $\boldsymbol{S} \in \mathbb{R}^{m \times c}$ called the *seed matrix*. For our example, the seed matrix is

$$\boldsymbol{S} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \end{pmatrix}, \tag{B.9}$$

We can then get a compressed Hessian by computing the product $\boldsymbol{J}(\boldsymbol{x})\boldsymbol{S}$. For our example, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{S} = \begin{pmatrix} 2x_1 & 1 \\ 2x_3 & \cos(x_2) \\ e^{x_4} & 0 \end{pmatrix}. \tag{B.10}$$

This compressed matrix contains all the information needed to recover the original Jacobian. Instead of storing a full 3×4 Jacobian, we only need to store a 3×2 compressed matrix. To recover the original $\boldsymbol{J}$, we can use the coloring information to unpack the compressed matrix. The first compressed column supplies the blue entries, and the second supplies the orange entries:

$$
\begin{aligned}
J_{11} = Y_{11} &= 2x_1, & J_{12} = Y_{12} &= 1, \\
J_{23} = Y_{21} &= 2x_3, & J_{22} = Y_{22} &= \cos(x_2), \\
J_{34} = Y_{31} &= e^{x_4}.
\end{aligned} \tag{B.11}
$$

The sparsity pattern places these values in their original columns and sets all other entries to zero:

$$
\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \\ 0 & \cos(x_2) & 2x_3 & 0 \\ 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{B.12}
$$

Recovering $\boldsymbol{J}$ from $\boldsymbol{J}(\boldsymbol{x})\boldsymbol{S}$ is called *decompression*.

B.5 The Hessian and HVPs

For a function $f : \mathbb{R}^m \to \mathbb{R}$, the Hessian matrix is defined as

$$
\boldsymbol{H}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_m \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_m^2} \end{pmatrix} \in \mathbb{R}^{m \times m}. \tag{B.13}
$$

The methods we have discussed for the Jacobian can also be applied to the Hessian. In fact, it can be applied to any sparse matrix, regardless of whether it is a Jacobian or Hessian. Therefore, the same coloring and decompression techniques can be used to recover the Hessian with fewer evaluations. Since the Hessian is symmetric, we can exploit this property to further reduce the number of evaluations needed. Furthermore, computing the Hessian-vector product (HVP) is equivalent to computing the vector-Hessian product (VHP) due to the symmetry of the Hessian. For instance, suppose we have a function $f : \mathbb{R}^4 \to \mathbb{R}$ defined as

$$
f(\boldsymbol{x}) = x_1^2 + x_1 x_2 + x_3^2 + x_4^2, \tag{B.14}
$$

Its Hessian is given by

$$
\boldsymbol{H}(\boldsymbol{x}) = \begin{pmatrix} 2 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}. \tag{B.15}
$$

Using the same graph coloring technique as before, we can find a coloring for the Hessian sparsity pattern. Columns 1 and 2 are the only ones that have a nonzero entry in the same row, so they cannot be assigned the same color. So $\chi(G) = 2$ and we can again assign blue to columns 1, 3, and 4 and orange to column 2. Colors follow columns, so the symmetric entries H_{12} and H_{21} have different colors despite having the same value. Therefore, we can get the seed matrix $\boldsymbol{S}$ as

$$\boldsymbol{S} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \end{pmatrix}. \tag{B.16}$$

Then we can get the compressed Hessian by computing the product $\boldsymbol{H}(\boldsymbol{x})\boldsymbol{S}$

$$\boldsymbol{H}(\boldsymbol{x})\boldsymbol{S} = \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ 2 & 0 \\ 2 & 0 \end{pmatrix}. \tag{B.17}$$

To recover the original Hessian, we can use the coloring information to unpack the compressed matrix

$$\begin{aligned} H_{11} = Y_{11} = 2, \qquad & H_{12} = Y_{12} = 1, \\ H_{21} = Y_{21} = 1, \qquad & H_{33} = Y_{31} = 2, \\ H_{44} = Y_{41} = 2. & \end{aligned} \tag{B.18}$$

All other coefficients are structural zeros.

B.6 ASD in PET

Since the potential energy computed by PET is the sum of the contributions from each atomic environment A_i, an atom i can only interact with its neighbors in A_i. Therefore, when computing the second derivatives of the potential, only atoms inside A_i can contribute to the Hessian entries $H_{i\alpha,j\beta}$, the other entries are structural zeros. The Hessian can be written as blocks of 3×3 matrices, where each block corresponds to the second derivatives of the potential with respect to the coordinates of two atoms i and j:

$$H_{i\alpha,j\beta} = \sum_t \frac{\partial^2 \varepsilon_t}{\partial R_{i\alpha} \partial R_{j\beta}}. \tag{B.19}$$

We can represent the pairs of atoms that interact with each other as a graph, where each atom is a vertex and an edge connects two atoms if they are neighbors in the same atomic environment. A compact way of visualizing these interactions is by using an adjacency matrix $\boldsymbol{A}$, where $A_{ij} = 1$ if atoms i and j are connected, and 0 otherwise. It is important to construct this matrix, because by doing so we can identify the nonzero entries of the Hessian and thus use the graph coloring technique to detect the sparsity pattern of the Hessian, without having to compute the Hessian explicitly. With this graph, we can identify paths between different atoms, even if they are not directly connected. This is achieved by traversing intermediate nodes, which allows us to find all pairs of atoms that are connected by a path of length k or less. The number of nodes to get from one atom to another is called a hop. We can determine the k-hop pattern by computing the powers of the adjacency matrix $\boldsymbol{A}$.

$$\boldsymbol{A}^{(k)} = \bigvee_{i=1}^{k} \boldsymbol{A}^i \tag{B.20}$$

The coefficients $\boldsymbol{A}_{ij}^{(k)}$ are 1 if atoms i and j are connected by a path of length k or less, and 0 otherwise. For instance, if $(\boldsymbol{A}^{(2)})_{ij} = 1$, atoms i and j are connected by a path of length 2 or less.

B.7 Hessian truncation

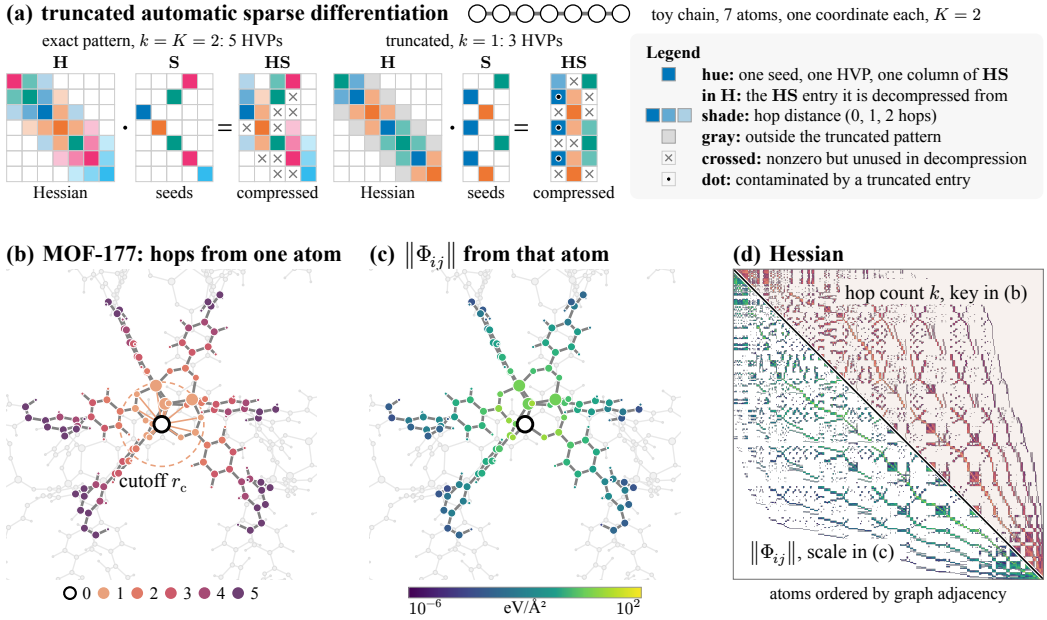


Figure 13: Truncation error in the Hessian. The blue blocks are retained, and the orange blocks are discarded. The retained blocks are contaminated by the omitted couplings, which are still present in the energy function. Figure adapted from [**?**].

The number of hops K needed to recover the full Hessian is given by $K = 2L + 1$, where L is the number message-passing layers in the PET architecture. For instance, PET-MAD-1.5-S has $L = 3$ message-passing layers, so we need $K = 7$ hops to recover the full Hessian. Therefore, $\boldsymbol{A}^{(7)}$ tell us which entries are nonzero and we can use it to find the coloring of the Hessian. The higher K is, the more nonzero entries we have in the Hessian, and the more evaluations we need to recover it. If we truncate the Hessian to a smaller number of hops $k < K$, $\boldsymbol{A}^{(k)}$ becomes sparser and fewer HVPs are needed to recover the truncated Hessian. It has been shown in [43] that this is a reasonable approximation, since the Hessian entries decay with distance and it allows for a more efficient computation. We can see this in Fig. 13 on plot (b), where we have the atoms around the central atom, each having a color corresponding to the number of hops from the central atom. We see that all the one hop neighbors are located inside the cutoff radius. The further away we are from the central atom, the higher the number of hops. In the plot (c), we have the frobenius norm of the Hessian blocks for the previous example between the central atom and its neighbors. It is clear from this plot that the norm decays with the number of hops, thus justifying the truncation of the Hessian.

References

[1] C. Cohen-Tannoudji, B. Diu, and F. Laloë, *Quantum Mechanics, Volume I: Basic Concepts, Tools, and Applications*, 2nd ed., Wiley-VCH, Weinheim (2020), publisher edition.

[2] M. Born and R. Oppenheimer, *Zur Quantentheorie der Molekeln*, Annalen der Physik **84**, 457–484 (1927), doi:10.1002/andp.19270840804.

[3] M. P. Allen and D. J. Tildesley, *Computer Simulation of Liquids*, 2nd ed., Oxford University Press (2017), publisher edition.

[4] M. E. Tuckerman, *Statistical Mechanics: Theory and Molecular Simulation*, 2nd ed., Oxford University Press (2023), publisher edition.

[5] R. M. Martin, *Electronic Structure: Basic Theory and Practical Methods*, 2nd ed., Cambridge University Press (2004), doi:10.1017/CBO9780511805769.

[6] P. Hohenberg and W. Kohn, *Inhomogeneous Electron Gas*, Physical Review **136**, B864–B871 (1964), doi:10.1103/PhysRev.136.B864.

[7] W. Kohn and L. J. Sham, *Self-Consistent Equations Including Exchange and Correlation Effects*, Physical Review **140**, A1133–A1138 (1965), doi:10.1103/PhysRev.140.A1133.

[8] N. Hanikel *et al.*, *Evolution of Water Structures in Metal-Organic Frameworks for Improved Atmospheric Water Harvesting*, Science **374**, 454–459 (2021), doi:10.1126/science.abj0890.

[9] C. L. Hobday *et al.*, *Understanding the Adsorption Process in ZIF-8 Using High Pressure Crystallography and Computational Modelling*, Nature Communications **9**, 1429 (2018), doi:10.1038/s41467-018-03878-6.

[10] C. Rowley, *Periodic Boundary Conditions in 2D* (2015), Wikimedia Commons, CC BY-SA 4.0.

[11] LAMMPS documentation, *compute pressure command*, online documentation (accessed 2 October 2026).

[12] LAMMPS documentation, *fix npt command*, online documentation (accessed 2 November 2026).

[13] LAMMPS developers, *Nosé–Hoover integrator implementation*, `FixNH::compute_scalar`, source code (accessed 15 October 2026).

[14] LAMMPS documentation, *min_style fire command*, online documentation (accessed 13 October 2026).

[15] H. Flyvbjerg and H. G. Petersen, *Error Estimates on Averages of Correlated Data*, Journal of Chemical Physics **91**, 461–466 (1989), doi:10.1063/1.457480.

[16] M. Ceriotti, M. Parrinello, T. E. Markland, and D. E. Manolopoulos, *Efficient Stochastic Thermostatting of Path Integral Molecular Dynamics*, Journal of Chemical Physics **133**, 124104 (2010), doi:10.1063/1.3489925.

[17] R. Diestel, *Graph Theory*, 6th ed., Graduate Texts in Mathematics **173**, Springer (2025), author's textbook site.

[18] A. H. Gebremedhin, A. Pothen, A. Tarafdar, and A. Walther, *Efficient Computation of Sparse Hessians Using Coloring and Automatic Differentiation*, INFORMS Journal on Computing **21**, 209–223 (2009), doi:10.1287/ijoc.1080.0286.

[19] A. Hill, *Graph Coloring*, ASDEX documentation, online documentation.

[20] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković, *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*, arXiv:2104.13478 (2021), arXiv:2104.13478.

[21] V. Kapil *et al.*, *Modeling the Structural and Thermal Properties of Loaded Metal–Organic Frameworks: An Interplay of Quantum and Anharmonic Fluctuations*, Journal of Chemical Theory and Computation **15**, 3237–3249 (2019), doi:10.1021/acs.jctc.8b01297.

[22] A. E. Abu El-maaty, O. Saoudi, A. S. Abdelrazik, and R. Ben-Mansour, *Experimental investigation of specific heat and thermal stability of MOF-303 at different water vapor loading and various temperatures*, International Communications in Heat and Mass Transfer **165**, 109088 (2025), doi:10.1016/j.icheatmasstransfer.2025.109088.

[23] Y. Ming, J. Purewal, D. Liu, A. Sudik, C. Xu, J. Yang, M. Veenstra, K. Rhodes, R. Soltis, J. Warner, M. Gaab, U. Müller, and D. J. Siegel, *Thermophysical properties of MOF-5 powders*, Microporous and Mesoporous Materials **185**, 235–244 (2014), doi:10.1016/j.micromeso.2013.11.015.

[24] P. F. Rosen, J. J. Calvin, M. S. Dickson, A. D. Katsenis, T. Friščić, A. Navrotsky, N. L. Ross, A. I. Kolesnikov, and B. F. Woodfield, *Heat capacity and thermodynamic functions of crystalline forms of the metal-organic framework zinc 2-methylimidazolate, Zn(MeIm)$_2$*, The Journal of Chemical Thermodynamics **136**, 160–169 (2019), doi:10.1016/j.jct.2019.05.008.

[25] P. F. Rosen, M. S. Dickson, J. J. Calvin, N. L. Ross, T. Friščić, A. Navrotsky, and B. F. Woodfield, *Thermodynamic Evidence of Structural Transformations in CO_2-Loaded Metal–Organic Framework Zn(MeIm)$_2$ from Heat Capacity Measurements*, Journal of the American Chemical Society **142**, 4833–4841 (2020), doi:10.1021/jacs.9b13883. Experimental values and uncertainties are from Supporting Information Table 3, doi:10.1021/jacs.9b13883.s001.

[26] A. Vaswani *et al.*, *Attention Is All You Need*, Advances in Neural Information Processing Systems **30** (2017), arXiv:1706.03762.

[27] T. B. Brown *et al.*, *Language Models are Few-Shot Learners*, Advances in Neural Information Processing Systems **33**, 1877–1901 (2020), arXiv:2005.14165.

[28] A. Mazitov *et al.*, *PET-MAD as a Lightweight Universal Interatomic Potential for Advanced Materials Modeling*, Nature Communications **16**, 10653 (2025), doi:10.1038/s41467-025-65662-7.

[29] C. Malosso *et al.*, *High-quality, high-information datasets for universal atomistic machine learning*, arXiv:2603.02089 (2026), arXiv:2603.02089.

[30] PET-SOL local checkpoint, `pet_sol-s-best_nostress.ckpt`, archived model and training metadata in the project `models/` directory (inspected 3 November 2026).

[31] S. N. Pozdnyakov and M. Ceriotti, *Smooth, Exact Rotational Symmetrization for Deep Learning on Point Clouds*, Advances in Neural Information Processing Systems **36** (2023), conference paper; the supplied article is arXiv:2305.19302v3 (6 February 2024), including Appendices A–H.

[32] L. Vanduyfhuys *et al.*, *QuickFF: A Program for a Quick and Easy Derivation of Force Fields for Metal–Organic Frameworks from Ab Initio Input*, Journal of Computational Chemistry **36**, 1015–1027 (2015).

[33] G. J. Martyna, D. J. Tobias, and M. L. Klein, *Constant Pressure Molecular Dynamics Algorithms*, Journal of Chemical Physics **101**, 4177–4189 (1994), doi:10.1063/1.467468.

[34] W. Shinoda, M. Shiga, and M. Mikami, *Rapid estimation of elastic constants by molecular dynamics simulation under constant stress*, Physical Review B **69**, 134103 (2004), doi:10.1103/PhysRevB.69.134103.

[35] S. Nosé, *A unified formulation of the constant temperature molecular dynamics methods*, Journal of Chemical Physics **81**, 511–519 (1984), doi:10.1063/1.447334.

[36] W. G. Hoover, *Canonical dynamics: Equilibrium phase-space distributions*, Physical Review A **31**, 1695–1697 (1985), doi:10.1103/PhysRevA.31.1695.

[37] G. J. Martyna, M. L. Klein, and M. Tuckerman, *Nosé–Hoover Chains: The Canonical Ensemble via Continuous Dynamics*, Journal of Chemical Physics **97**, 2635–2643 (1992), doi:10.1063/1.463940.

[38] F. Bigi, S. Chong, M. Ceriotti, and F. Grasselli, *A prediction rigidity formalism for low-cost uncertainties in trained neural networks*, Machine Learning: Science and Technology **5**, 045018 (2024), doi:10.1088/2632-2153/ad805f.

[39] M. Kellner and M. Ceriotti, *Uncertainty Quantification by Direct Propagation of Shallow Ensembles*, Machine Learning: Science and Technology **5**, 035006 (2024), doi:10.1088/2632-2153/ad594a.

[40] G. Imbalzano *et al.*, *Uncertainty estimation for molecular dynamics and sampling*, Journal of Chemical Physics **154**, 074102 (2021), doi:10.1063/5.0036522.

[41] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, *Automatic Differentiation in Machine Learning: A Survey*, Journal of Machine Learning Research **18**, 1–43 (2018), JMLR 18(153).

[42] A. Hill, G. Dalle, and A. Montoison, *An Illustrated Guide to Automatic Sparse Differentiation*, ICLR Blogposts (2025), online article.

[43] M. F. Langer, A. Hill, and M. Ceriotti, *Truncated automatic sparse differentiation for machine learning interatomic potentials*, arXiv:2609.20510 (2026), doi:10.48550/arXiv.2609.20510.